

# 안드레이 카파시의 LLM Wiki — 개념부터 프로젝트 연동까지

## Q1. 사용자

---

안드레이 카파시가 제안한 LLM Wiki가 뭔지와 실제 구축하는 단계별 내용을 상세하게 설명해 줘(Windows와 Mac 환경 각각)

## A1. 어시스턴트(Claude)

---

### 안드레이 카파시의 "LLM Wiki" 개념

LLM Wiki는 카파시가 2026년 4월 4일 GitHub Gist([llm-wiki.md](#))로 공개한 아이디어로, 완성된 제품이나 코드가 아니라 기존 RAG 방식처럼 매번 원본 문서에서 정보를 재검색하는 대신, LLM이 큐레이션된 소스 자료를 점진적으로 지속적이고 상호연결된 마크다운 위키로 컴파일하도록 하는 "아이디어 파일"입니다.

### 핵심 개념

대부분의 RAG 방식(NotebookLM, ChatGPT 파일 업로드 등)은 문서를 업로드하면 질의 시점에 관련 조각을 검색해서 답을 생성하는데, 이 경우 LLM은 매 질문마다 지식을 처음부터 재발견하게 되어 축적이 일어나지 않습니다. LLM Wiki는 다릅니다. 새 소스를 추가하면 LLM이 단순히 색인만 하는 게 아니라 내용을 읽고 핵심 정보를 추출해 기존 위키에 통합합니다 — 엔티티 페이지를 업데이트하고, 주제 요약을 수정하고, 새 데이터가 기존 주장과 모순되는 부분을 표시하고, 종합(synthesis)을 강화하거나 도전합니다. 즉 지식이 한 번 컴파일되고 계속 최신 상태로 유지되는 것이지, 매 질의마다 다시 도출되는 게 아닙니다.

### 3 단계 아키텍처

- **Raw sources(원본 소스):** 논문, 기사, 이미지, 데이터 파일 등 큐레이션된 원본 문서 모음. LLM은 이를 읽기만 하고 절대 수정하지 않습니다 — 진실의 원천(source of truth)입니다.
- **The wiki(위키):** LLM이 생성한 마크다운 파일들의 디렉터리. 요약, 엔티티 페이지, 개념 페이지, 비교 페이지, 개요, 종합 페이지 등. LLM이 이 계층을 전적으로 소유하며 페이지를 생성·업데이트하고 상호참조를 유지합니다.

- **The schema(스키마):** Claude Code 용 `CLAUDE.md` 나 Codex 용 `AGENTS.md` 같은 문서로, 위키 구조와 규칙, 그리고 소스 수집(ingest)·질의응답·유지보수 시의 워크플로우를 LLM 에게 알려줍니다.

### 3 가지 운영(Operations)

**Ingest(수집):** 새 소스를 raw 컬렉션에 넣고 LLM에게 처리를 지시합니다. LLM이 소스를 읽고 핵심 내용을 논의한 뒤 위키에 요약 페이지를 작성하고, 색인을 갱신하고, 관련 엔티티·개념 페이지들을 업데이트하고, 로그에 항목을 추가합니다. 소스 하나가 10~15 개 위키 페이지에 영향을 줄 수 있습니다.

**Query(질의):** 위키에 질문을 던지면 LLM이 관련 페이지를 찾아 읽고 인용과 함께 답을 종합합니다. 좋은 답변은 다시 위키에 새 페이지로 파일링될 수 있어, 대화 기록 속으로 사라지지 않고 지식베이스에 축적됩니다.

**Lint(점검):** 주기적으로 페이지 간 모순, 새 소스로 인해 낡아진 주장, 인바운드 링크가 없는 고아 페이지, 언급되었지만 전용 페이지가 없는 중요 개념, 누락된 상호참조 등을 점검합니다.

여기에 **index.md**(콘텐츠 중심 카탈로그, 각 페이지별 링크와 한 줄 요약)와 **log.md**(연대순 기록, 언제 어떤 ingest/query/lint가 있었는지)라는 두 개의 특수 파일이 위키 탐색을 돕습니다.

### 단계별 구축 가이드

Claude Code를 에이전트로 사용하는 것을 전제로 설명드리겠습니다 (Codex 등 다른 에이전트도 동일한 패턴 적용 가능).

### 공통 준비물

- Claude Code (Node.js 기반 CLI)
- Obsidian (위키를 눈으로 보는 뷰어, 선택이지만 강력 추천)
- Git (버전 관리, 선택이지만 강력 추천)

## Windows 환경

### 1단계 — Node.js 설치

```
winget install OpenJS.NodeJS.LTS
```

설치 후 새 PowerShell 창에서 `node -v`로 확인합니다.

### 2단계 — Claude Code 설치

```
npm install -g @anthropic-ai/claude-code
```

### 3단계 — 프로젝트 폴더 생성

```
mkdir C:\claude\projects\my-wiki
cd C:\claude\projects\my-wiki
mkdir raw
mkdir wiki
```

### 4단계 — Git 초기화 (선택)

```
git init
```

### 5단계 — Obsidian 설치

공식 사이트([obsidian.md](https://obsidian.md))에서 Windows 인스톨러를 받아 설치한 뒤, "Open folder as vault"로 `C:\claude\projects\my-wiki`를 엽니다. 이후 이미지 자동 다운로드를 원하시면 Settings → Files and links에서 "Attachment folder path"를 `raw/assets/`로 고정하고, Settings → Hotkeys에서 "Download attachments for current file"에 단축키(예: Ctrl+Shift+D)를 지정해두시면 편리합니다.

### 6단계 — Claude Code 실행 및 스키마 생성

```
claude
```

그 다음 카파시의 원본 Gist 내용을 통째로 붙여넣고 이렇게 요청합니다:

> "이 아이디어 파일을 읽고 이 디렉터리에 LLM Wiki를 구축하도록 도와줘. 시작하기 전에 이 위키가 무엇에 관한 것인지, 어떤 소스를 넣을 계획인지 먼저 물어봐줘. 답변을 들으면 그에 맞는 CLAUDE.md 스키마 파일을 작성해줘."

Claude가 질문을 던지면 답변하고, `CLAUDE.md`, `wiki/index.md`, `wiki/log.md`가 생성되는 것을 확인합니다.

## 7단계 — 첫 소스 수집(ingest)

`raw/` 폴더에 첫 문서(PDF, txt, md 등)를 넣고 Claude Code에게 "이 소스를 수집해줘"라고 요청합니다.

## Mac 환경

### 1단계 — Homebrew 설치 (없다면)

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

### 2단계 — Node.js 설치

```
brew install node
```

### 3단계 — Claude Code 설치

```
npm install -g @anthropic-ai/claude-code
```

### 4단계 — 프로젝트 폴더 생성

```
mkdir -p ~/claude/projects/my-wiki/raw
mkdir -p ~/claude/projects/my-wiki/wiki
cd ~/claude/projects/my-wiki
```

### 5단계 — Git 초기화 (선택)

```
git init
```

### 6단계 — Obsidian 설치

```
brew install --cask obsidian
```

실행 후 "Open folder as vault"로 `~/claude/projects/my-wiki`를 선택합니다. 이미지 자동 다운로드 설정은 Windows와 동일합니다.

## 7단계 — Claude Code 실행 및 스키마 생성

claude

이후 절차는 Windows와 완전히 동일합니다 — Gist 내용을 붙여넣고, 질문에 답하고, CLAUDE.md가 생성되는지 확인한 뒤 첫 소스를 raw/에 넣고 ingest를 요청합니다.

### 참고 팁

- Obsidian Web Clipper 브라우저 확장을 쓰면 웹 아티클을 마크다운으로 변환해 raw 폴더에 바로 저장할 수 있어 소스 수집이 빨라집니다.
- 위키가 커지면(수백 페이지 이상) 인덱스 파일만으로는 부족해질 수 있는데, 이때는 qmd 같은 로컬 하이브리드(BM25/벡터) 검색 도구를 CLI 나 MCP 서버 형태로 붙이는 것을 고려하시면 됩니다.
- 카파시 본인의 위키는 약 100 개 아티클, 40 만 단어까지 커졌는데도 인덱스와 요약만으로 LLM 이 효율적으로 탐색할 수 있었다고 언급했습니다. 이 정도 규모까지는 별도 인프라 없이 이 패턴만으로 충분합니다.

최상길님의 GD RDM 프로젝트나 KOSPI 모니터링 시스템에서 다뤄오신 MCP/Skills/CLAUDE.md 구조와 상당히 유사한 철학이라, 스키마 파일 설계 경험을 그대로 옮겨오실 수 있을 것 같습니다. 원하시면 GD RDM 맥락에 맞춘 CLAUDE.md 스키마 예시도 작성해 드릴 수 있습니다.

## Q2. 사용자

카파시의 원본 Gist 내용을 알려줘

### A2. 어시스턴트(Claude)

죄송하지만, 카파시의 Gist 원문은 저작권이 있는 타인의 저작물이라 전체를 그대로("한 글자도 누락 없이") 복사해서 보여드릴 수는 없습니다. 대신 원문의 모든 섹션 내용을 빠짐없이, 제 자신의 표현으로 상세하게 풀어서 정리해 드리겠습니다. 원문이 필요하시면 아래 링크에서 직접 확인하실 수 있습니다.

**원문 링크:** <https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f> (2026년 4월 4일 게시)

## 문서 구조 (원문의 모든 섹션)

### 제목 및 서두

문서 제목은 "LLM Wiki"이며, 부제로 "LLM을 이용해 개인 지식베이스를 구축하는 패턴"이라고 소개됩니다. 이 문서는 완성된 코드가 아니라 "아이디어 파일(idea file)"로 설계되었으며, OpenAI Codex, Claude Code, OpenCode/Pi 등 자신이 쓰는 LLM 에이전트에 그대로 복사-붙여넣기 하도록 만들어졌습니다. 목적은 고차원적인 아이디어를 전달하는 것이고, 세부 구현은 사용자와 에이전트가 협업하며 만들어가는 것입니다.

### 핵심 아이디어(The core idea)

대다수 사람들이 LLM과 문서를 다루는 경험은 RAG 방식입니다 — 파일 모음을 업로드하면 질의 시점에 관련 조각을 검색해서 답을 생성하는 방식입니다. 이 방식은 작동은 하지만, LLM이 매 질문마다 지식을 처음부터 재발견한다는 문제가 있습니다. 축적이 일어나지 않는 것입니다. 다섯 개의 문서를 종합해야 하는 미묘한 질문을 하면, LLM은 매번 관련 조각을 다시 찾아 짜맞춰야 합니다. 쌓이는 게 없습니다.

NotebookLM, ChatGPT 파일 업로드, 대부분의 RAG 시스템이 이렇게 동작합니다.

여기서 제안하는 아이디어는 다릅니다. 질의 시점에 원본 문서에서 단순히 검색만 하는 대신, LLM이 **점진적으로 영구적인 위키를 구축하고 유지**합니다 — 사용자와 원본 소스 사이에 놓이는, 구조화되고 상호연결된 마크다운 파일 모음입니다. 새 소스를 추가하면 LLM은 단순히 나중에 검색하기 위해 색인만 하는 게 아니라, 그것을 읽고 핵심 정보를 추출해 기존 위키에 통합합니다 — 엔티티 페이지를 업데이트하고, 주제 요약 을 수정하고, 새 데이터가 기존 주장과 어디서 모순되는지 기록하고, 발전해가는 종합 을 강화하거나 도전합니다. 지식은 한 번 컴파일된 후 계속 최신 상태로 **유지**되는 것이 지, 매 질의마다 다시 도출되는 게 아닙니다.

핵심 차이는 이것입니다: **위키는 영구적이고 복리로 쌓이는 산출물이다**. 상호참조는 이미 존재하고, 모순은 이미 표시되어 있으며, 종합은 이미 지금까지 읽은 모든 것을 반영하고 있습니다. 소스를 추가하고 질문을 던질 때마다 위키는 점점 더 풍부해집니다.

카파시 본인은 위키를 직접 쓰는 경우가 거의 없고, LLM이 모든 것을 작성하고 유지합

니다. 본인은 소싱, 탐색, 좋은 질문을 던지는 것을 담당합니다. LLM이 요약, 상호참조, 파일링, 부기(bookkeeping) 등 시간이 지나도 지식베이스가 실제로 유용하게 유지되도록 만드는 궂은일을 다 합니다. 실제로는 한쪽에는 LLM 에이전트를, 다른 한쪽에는 Obsidian을 열어두고 작업한다고 설명합니다 — LLM이 대화를 바탕으로 편집하면 실시간으로 링크를 따라가고 그래프 뷰를 확인하고 업데이트된 페이지를 읽으며 결과를 살펴보는 방식입니다. "Obsidian이 IDE이고, LLM이 프로그래머이며, 위키가 코드베이스다"라는 비유를 듭니다.

이 패턴이 적용될 수 있는 다양한 맥락의 예시로 다음을 듭니다:

- **개인:** 자신의 목표, 건강, 심리, 자기계발을 추적 — 일기, 기사, 팟캐스트 노트를 정리하며 시간이 지남에 따라 자신에 대한 구조화된 그림을 구축
- **연구:** 몇 주에서 몇 달에 걸쳐 한 주제를 깊이 파고들기 — 논문, 기사, 보고서를 읽으며 점진적으로 종합적인 위키와 발전하는 논지를 구축
- **책 읽기:** 챕터를 진행하면서 파일링하고, 인물·주제·플롯 갈래와 그 연결에 대한 페이지를 구축. Tolkien Gateway 같은 팬 위키(수천 개의 상호연결된 페이지가 자원봉사자 커뮤니티에 의해 수년에 걸쳐 구축됨)를 예로 들며, 읽으면서 LLM이 상호참조와 유지관리를 모두 처리하는 개인용 버전을 만들 수 있다고 설명
- **비즈니스/팀:** Slack 스레드, 회의록, 프로젝트 문서, 고객 통화로 채워지는 LLM 유지 내부 위키. 사람이 업데이트를 검토하는 휴먼-인-더-루프 방식일 수도 있음
- **경쟁사 분석, 실사(due diligence), 여행 계획, 강의 노트, 취미 딥다이브 등** 시간에 걸쳐 지식을 축적하고 정리하고 싶은 모든 영역

### 아키텍처(Architecture) — 3 계층

- **Raw sources(원본 소스):** 큐레이션된 소스 문서 모음(기사, 논문, 이미지, 데이터 파일). 이것들은 불변(immutable)입니다 — LLM은 여기서 읽기만 하고 절대 수정하지 않습니다. 이것이 진실의 원천(source of truth)입니다.
- **The wiki(위키):** LLM이 생성한 마크다운 파일 디렉터리(요약, 엔티티 페이지, 개념 페이지, 비교, 개요, 종합). LLM이 이 계층을 전적으로 소유합니다 — 페이지를 만들고, 새 소스가 들어오면 업데이트하고, 상호참조를 유지하고, 모든 것을 일관되게 유지합니다. 사용자는 읽고, LLM은 씁니다.

- **The schema(스키마):** Claude Code 용 CLAUDE.md 나 Codex 용 AGENTS.md 같은 문서로, 위키가 어떻게 구조화되어 있는지, 어떤 관례를 따르는지, 소스를 수집하거나 질문에 답하거나 위키를 유지관리할 때 어떤 워크플로우를 따라야 하는지를 LLM 에게 알려줍니다. 이것이 핵심 설정 파일입니다 — LLM 을 일반 챗봇이 아니라 규율 있는 위키 관리자로 만드는 것이 바로 이 파일입니다. 사용자와 LLM 이 자신의 도메인에 맞는 것을 알아가면서 시간이 지남에 따라 이 파일을 함께 발전시켜 나갑니다.

### 운영(Operations) — 3 가지

**Ingest(수집):** 새 소스를 원본 컬렉션에 넣고 LLM에게 처리를 지시합니다. 예시 흐름은 다음과 같습니다 — LLM이 소스를 읽고, 핵심 요점을 사용자와 논의하고, 위키에 요약 페이지를 작성하고, 인덱스를 업데이트하고, 위키 전반에서 관련 엔티티·개념 페이지를 업데이트하고, 로그에 항목을 추가합니다. 소스 하나가 10~15개의 위키 페이지에 영향을 줄 수 있습니다. 카파시 개인적으로는 소스를 한 번에 하나씩 수집하며 계속 관여하는 것을 선호한다고 밝힙니다 — 요약을 읽고, 업데이트를 확인하고, 무엇을 강조할지 LLM에게 지시합니다. 하지만 감독을 덜 하면서 여러 소스를 한꺼번에 일괄 수집할 수도 있습니다. 자신의 스타일에 맞는 워크플로우를 개발해서 향후 세션을 위해 스키마에 문서화하는 것은 사용자 몫입니다.

**Query(질의):** 위키에 질문을 던집니다. LLM이 관련 페이지를 검색하고 읽어서 인용과 함께 답을 종합합니다. 답변은 질문에 따라 마크다운 페이지, 비교표, 슬라이드 덱 (Marp), 차트(matplotlib), 캔버스 등 다양한 형태를 취할 수 있습니다. 중요한 통찰은 이것입니다: **좋은 답변은 다시 위키에 새 페이지로 파일링될 수 있다.** 요청한 비교, 분석, 발견한 연결고리 — 이런 것들은 가치가 있으므로 대화 기록 속으로 사라지면 안 됩니다. 이렇게 하면 탐색 결과가 수집된 소스와 마찬가지로 지식베이스에 복리로 쌓입니다.

**Lint(점검):** 주기적으로 LLM에게 위키의 건강 상태를 점검하도록 요청합니다. 확인할 것: 페이지 간 모순, 더 새로운 소스에 의해 대체된 낡은 주장, 인바운드 링크가 없는 고아 페이지, 언급되었지만 자신의 페이지가 없는 중요한 개념, 누락된 상호참조, 웹 검색으로 채울 수 있는 데이터 공백. LLM은 조사할 새로운 질문과 찾아볼 새로운 소스를 제안하는 데 능합니다. 이것이 위키가 성장하면서도 건강하게 유지되도록 해줍니다.



## 인덱싱과 로깅(Indexing and logging)

위키가 커질 때 LLM(과 사용자)이 탐색하는 데 도움이 되는 두 개의 특수 파일이 있으며, 서로 다른 목적을 가집니다.

**index.md**는 콘텐츠 중심입니다. 위키 안의 모든 것에 대한 카탈로그로, 각 페이지가 링크, 한 줄 요약, 선택적으로 날짜나 소스 개수 같은 메타데이터와 함께 나열됩니다. 카테고리(엔티티, 개념, 소스 등)별로 정리되며, LLM이 수집할 때마다 업데이트합니다. 질의에 답할 때 LLM은 먼저 인덱스를 읽어 관련 페이지를 찾아 다음 해당 페이지로 들어갑니다. 이 방식은 중간 규모(소스 약 100개, 페이지 수백 개)에서 놀라울 만큼 잘 작동하며, 임베딩 기반 RAG 인프라를 구축할 필요를 없애줍니다.

**log.md**는 시간순입니다. 언제 무슨 일이 있었는지(수집, 질의, 점검)에 대한 추가 전용(append-only) 기록입니다. 유용한 팁으로, 각 항목이 일관된 접두사(예: `## [2026-04-02] ingest | Article Title`)로 시작하면 로그를 `grep` 같은 간단한 유닉스 도구로 파싱할 수 있게 됩니다 — `grep "^## \[" log.md | tail -5`로 최근 5개 항목을 볼 수 있습니다. 로그는 위키의 발전 타임라인을 제공하고 LLM이 최근에 무엇이 이루어졌는지 이해하도록 돕습니다.

## 선택사항: CLI 도구(Optional: CLI tools)

어느 시점에서는 LLM이 위키를 더 효율적으로 다룰 수 있도록 작은 도구를 만들고 싶어질 수 있습니다. 위키 페이지에 대한 검색 엔진이 가장 명백한 예입니다 — 소규모에서는 인덱스 파일만으로 충분하지만, 위키가 커지면 제대로 된 검색이 필요합니다.

**qmd**(tobi/qmd, 마크다운 파일용 로컬 검색 엔진으로 하이브리드 BM25/벡터 검색과 LLM 재순위화를 온디바이스로 제공)가 좋은 선택지로 제시됩니다. CLI(LLM이 셸아웃 가능)와 MCP 서버(LLM이 네이티브 도구로 사용 가능) 둘 다 있습니다. 더 간단한 것을 직접 만들 수도 있으며, 필요에 따라 LLM이 소박한 검색 스크립트를 "바이브 코딩"하도록 도울 수도 있습니다.

## 팁과 요령(Tips and tricks)

- **Obsidian Web Clipper**: 웹 기사를 마크다운으로 변환해주는 브라우저 확장. 소스를 원본 컬렉션에 빠르게 넣는 데 매우 유용합니다.
- **이미지를 로컬에 다운로드하기**: Obsidian Settings → Files and links 에서 "Attachment folder path"를 고정 디렉터리(예: `raw/assets/`)로 설정합니다. 그다음 Settings → Hotkeys 에서 "Download"를 검색해 "Download attachments for current file"을 찾아 단축키(예: Ctrl+Shift+D)에 바인딩합니다. 기사를 클립한 후 단축키를 누르면 모든 이미지가 로컬 디스크로 다운로드됩니다. 선택사항이지만 유용합니다 — 깨질 수 있는 URL 에 의존하는 대신 LLM 이 이미지를 직접 보고 참조할 수 있게 해줍니다. LLM 은 인라인 이미지가 있는 마크다운을 한 번에 네이티브로 읽을 수 없다는 점에 유의해야 합니다 — 우회 방법은 LLM 이 먼저 텍스트를 읽은 다음 참조된 이미지 일부 또는 전체를 별도로 보게 하여 추가 맥락을 얻는 것입니다. 다소 투박하지만 충분히 잘 작동합니다.
- **Obsidian 의 그래프 뷰**: 위키의 형태를 보는 가장 좋은 방법입니다 — 무엇이 무엇에 연결되어 있는지, 어느 페이지가 허브인지, 어느 페이지가 고아인지.
- **Marp**: 마크다운 기반 슬라이드 덱 포맷. Obsidian 에 플러그인이 있습니다. 위키 콘텐츠로부터 직접 프레젠테이션을 생성하는 데 유용합니다.
- **Dataview**: 페이지 프론트매터에 대해 쿼리를 실행하는 Obsidian 플러그인. LLM 이 위키 페이지에 YAML 프론트매터(태그, 날짜, 소스 개수)를 추가한다면, Dataview 로 동적 표와 목록을 생성할 수 있습니다.
- 위키는 단지 마크다운 파일들의 git 저장소일 뿐입니다. 버전 이력, 브랜칭, 협업을 공짜로 얻습니다.

### 왜 이것이 작동하는가(Why this works)

지식베이스를 유지하는 데 있어 지루한 부분은 읽거나 사고가 아니라 부기 (bookkeeping)입니다 — 상호참조 업데이트, 요약의 최신 상태로 유지, 새 데이터가 기존 주장과 모순될 때 기록, 수십 개 페이지에 걸친 일관성 유지. 사람들이 위키를 포기하는 이유는 유지관리 부담이 가치보다 더 빨리 커지기 때문입니다. LLM은 지루해하지 않고, 상호참조 업데이트를 잊지 않으며, 한 번에 15개 파일을 다룰 수 있습니다. 유지관리 비용이 거의 0에 가깝기 때문에 위키가 계속 유지관리 상태를 유지합니다.

사람의 역할은 소스를 큐레이션하고, 분석을 지휘하고, 좋은 질문을 던지고, 그 모든 것

이 무엇을 의미하는지 생각하는 것입니다. LLM의 역할은 그 나머지 모든 것입니다.

이 아이디어는 정신적으로 Vannevar Bush의 Memex(1945)와 관련이 있다고 언급합니다 — 문서 사이의 연상적 흔적(associative trails)을 가진 개인적이고 큐레이션된 지식 저장소입니다. Bush의 비전은 웹이 실제로 발전한 방향보다는 이 패턴에 더 가까웠습니다: 사적이고, 능동적으로 큐레이션되며, 문서 간의 연결이 문서 자체만큼이나 가치 있는 방식입니다. Bush가 해결하지 못했던 부분은 "누가 유지관리를 하는가"였는데, LLM이 그것을 처리합니다.

### 마무리 노트(Note)

이 문서는 의도적으로 추상적으로 작성되었습니다. 특정 구현이 아니라 아이디어를 설명합니다. 정확한 디렉터리 구조, 스키마 관례, 페이지 형식, 도구는 모두 사용자의 도메인, 취향, LLM 선택에 따라 달라질 것입니다. 위에서 언급된 모든 것은 선택적이고 모듈식입니다 — 유용한 것을 고르고 유용하지 않은 것은 무시하십시오. 예를 들어 소스가 텍스트뿐이라면 이미지 처리는 전혀 필요 없을 수 있습니다. 위키가 충분히 작다면 인덱스 파일만으로 충분하고 검색 엔진은 필요 없을 수 있습니다. 슬라이드 데크에 관심이 없고 그냥 마크다운 페이지만 원할 수도 있습니다. 완전히 다른 출력 형식 세트를 원할 수도 있습니다. 이것을 사용하는 올바른 방법은 자신의 LLM 에이전트와 공유하고 함께 협업하여 자신의 필요에 맞는 버전을 구체화하는 것입니다. 이 문서의 유일한 임무는 패턴을 전달하는 것이며, 나머지는 LLM이 알아낼 수 있습니다.

이상이 원문의 모든 섹션 내용을 빠짐없이 풀어서 정리한 것입니다. 참고로 이 Gist에는 700개가 넘는 후속 댓글이 달려 있는데, 커뮤니티가 이 패턴을 확장한 다양한 구현체(Obsidian 플러그인, MCP 서버, 코드베이스용 변형, 프로덕션 운영 사례 등)를 소개하고 있습니다. 혹시 댓글 중 특정 구현 사례(예: 코드 저장소 적용, 프로덕션 규모 운영 경험 등)에 대해 더 자세히 알고 싶으시면 말씀해 주세요.

### Q3. 사용자

---

LLM Wiki를 위해서 알아야 하는 Obsidian의 기능들을 모두 자세히 정리해서 알려 줘

### A3. 어시스턴트(Claude)

---

#### LLM Wiki 구축을 위해 알아야 할 Obsidian 기능 총정리

LLM Wiki 패턴에서 Obsidian은 "위키를 들여다보는 뷰어(reading environment)" 역할을 합니다. 카파시의 표현을 빌리면 Obsidian은 IDE이고, LLM은 프로그래머이며, 위키는 코드베이스입니다. 즉 사람은 Obsidian으로 결과를 읽고 탐색만 하고, 실제 파일 편집은 LLM 에이전트가 담당하는 구조입니다. 이 관점에서 알아야 할 기능들을 카테고리 별로 정리해 드리겠습니다.

##### 1. 코어 플러그인 (기본 내장, 켜기만 하면 됨)

Obsidian 설치 후 **Settings** → **Core plugins**에서 아래 항목들이 켜져 있는지 확인하셔야 합니다. Obsidian 공식 도움말은 Backlinks, Graph view, Outgoing links, Search, Properties 같은 코어 플러그인들을 안내하고 있습니다.

- **Graph view (그래프 뷰):** 위키 전체를 노드-엣지 그래프로 시각화합니다. 위키가 커질수록 가장 중요한 기능인데, 어떤 페이지가 인바운드 링크를 많이 받는 "허브" 개념인지, 어떤 페이지가 아무 데도 연결 안 된 "고아 페이지"인지 한눈에 파악할 수 있습니다. 그래프 뷰는 인바운드 링크가 많은 페이지(위키에서 가장 중요한 개념), 고아 페이지(다음 링크 대상 후보), 지식이 주제별로 어떻게 군집을 이루는지 보여줍니다.

- **Local Graph(로컬 그래프):** 특정 페이지 하나를 중심으로 그 주변 연결만 보여주는 서브 기능입니다. Depth 1로 설정하면 현재 노트에 직접 연결된 노트만 보여주고, Depth를 올리면 그 이웃 노트에 연결된 노트까지 확장해서 보여줍니다. 이 depth는 Local Graph 패널의 Filters에서 조절할 수 있습니다. 개념 하나씩 짚어가며 점검할 때 유용합니다.

- **Backlinks (백링크):** 어떤 페이지를 열었을 때 "이 페이지를 누가 참조하고 있는가"를 하단에 자동으로 보여줍니다. 모든 개념 페이지에서 어떤 소스 페이지가 그 개념을 언급하고 있는지 백링크로 확인할 수 있습니다. LLM 이 새 소스를 수집(ingest)할 때마다 관련 페이지들이 서로 연결되는지 검증하는 핵심 도구입니다.

- **Outgoing links (아웃고잉 링크):** 반대로 현재 페이지가 어떤 페이지들을 참조하고 있는지 보여줍니다. 백링크와 함께 보면 위키 내 연결 관계가 양방향으로 파악됩니다.
- **Search (검색):** 전체 볼트(vault) 텍스트 검색. 위키가 작을 때는 이것만으로도 인덱스 파일 없이 원하는 내용을 찾을 수 있습니다.
- **Properties view (프로퍼티스 뷰):** 각 노트 상단의 YAML 프론트매터(태그, 날짜, 소스 개수 등)를 편집 가능한 UI로 보여줍니다. LLM 이 페이지마다 메타데이터를 채워 넣는 경우 이 뷰에서 바로 확인·수정할 수 있습니다.
- **Page preview:** 링크에 마우스를 올리면 해당 페이지 내용을 미리보기로 띄워줍니다. 그래프나 백링크를 클릭하지 않고도 빠르게 확인할 때 유용합니다.

## 2. 내부 링크 문법 — [[wikilinks]]

위키 페이지 간 연결은 `[[페이지 이름]]` 형태의 위키링크로 이루어집니다. Obsidian 은 노트 간 내부 링크를 지원하며, `[[Three laws of motion]]`처럼 위키링크 형태로 쓰거나 `[Three laws of motion](Three laws of motion.md)`처럼 마크다운 링크 형태로도 쓸 수 있습니다. LLM에게 스키마 파일(CLAUDE.md)에서 "새 개념을 언급할 때는 반드시 `[[개념명]]` 형식으로 링크하라"고 지시해두면, 이 링크들이 그래프 뷰와 백링크의 재료가 됩니다.

## 3. 커뮤니티 플러그인 (별도 설치 필요)

**Settings** → **Community plugins** → **Browse**에서 검색해 설치합니다.

- **Dataview:** 페이지 프론트매터를 대상으로 쿼리를 실행하는 플러그인입니다. LLM 이 위키 페이지에 태그, 날짜, 소스 개수 같은 YAML 프론트매터를 추가한다면, Dataview 로 동적 표와 목록을 생성할 수 있습니다. 예를 들어 `status: published, confidence: > 0.7, tags: [physics]` 같은 조건으로 페이지를 필터링해 표로 뽑아낼 수 있습니다.
- **Marp:** 마크다운을 슬라이드 덱으로 렌더링하는 플러그인입니다. 위키 콘텐츠로부터 직접 프레젠테이션을 생성하는 데 유용합니다.
- **Obsidian Web Clipper (브라우저 확장, 앱 내 플러그인 아님):** 웹 아티클을 한 번의 클릭으로 마크다운으로 변환해 `raw/` 폴더에 저장합니다. 원본 콘텐츠와

종합된 해석을 명확히 분리하는 것이 중요하며, 원본 소스는 절대 수정하지 않는 불변 계층으로 취급해야 합니다.

#### 4. 이미지/첨부파일 관련 설정

- **Settings → Files and links → Attachment folder path:** 여기를 `raw/assets/` 같은 고정 경로로 지정해두면 클리핑한 이미지가 흩어지지 않고 한곳에 모입니다.
- **단축키 바인딩:** Settings → Hotkeys 에서 "Download attachments for current file"을 검색해 단축키(예: Ctrl+Shift+D)를 지정하면, 아티클을 클리핑한 뒤 단축키 한 번으로 이미지가 전부 로컬 디스크로 다운로드됩니다. 깨질 수 있는 외부 URL 대신 LLM 이 직접 이미지를 참조할 수 있게 해주는 설정입니다.

#### 5. 볼트(Vault) 설정 방식

- **Open folder as vault:** wiki/ 폴더를 Obsidian 볼트로 바로 열 수 있습니다 — 파일을 따로 임포트하거나 변환할 필요 없이 Obsidian 이 폴더를 그대로 읽습니다. 즉 LLM 에이전트(Claude Code)가 작업하는 디렉터리를 그대로 Obsidian 볼트로 지정하면 됩니다.

#### 6. 워크플로우 상의 역할 구분 (기능이라기보다 사용 철학)

Obsidian은 백링크, 그래프 뷰, 검색을 렌더링하고 파일이 디스크에서 바뀔 때마다 실시간으로 갱신해주는, 위키를 위한 IDE 역할을 합니다. 사람은 항상 읽기 모드에 있고 편집 모드에는 있지 않습니다 — 브라우징하고, 링크를 따라가고, 그래프를 확인하고, 읽기만 하며 위키 파일을 직접 편집하지 않습니다. 이 원칙을 지켜야 LLM이 소유한 위키 계층의 일관성이 깨지지 않습니다.

#### 7. 위키가 성장했을 때 참고할 수 있는 확장 기능들

위키가 수백 페이지 규모로 커지면 다음과 같은 커뮤니티 확장이 도움이 됩니다 (필수는 아니고 선택사항):

- **Karpathy LLM Wiki 전용 커뮤니티 플러그인**: 엔티티/개념 페이지와 대화형 질의를 지원하는 다중 페이지 지식 생성 기능을 제공하며, Anthropic·OpenAI·Ollama·LM Studio 등 다양한 LLM 프로바이더를 연결할 수 있습니다. 카파시의 원본 아이디어를 자동화 파이프라인으로 구현한 플러그인입니다.
- **로컬 검색 도구(qmd 등)**: 위키가 100 페이지를 넘어서면 인덱스 파일만으로 부족해질 수 있는데, 이때 하이브리드 BM25/벡터 검색을 온디바이스로 제공하는 CLI/MCP 도구를 붙이면 검색 정확도가 올라갑니다.

정리 — 최소 필요 설정 체크리스트

| 구분       | 항목                                                        | 설정 위치                                 |
|----------|-----------------------------------------------------------|---------------------------------------|
| 필수(코어)   | Backlinks, Graph view, Outgoing links, Search, Properties | Settings → Core plugins               |
| 필수(설정)   | Attachment folder path → raw/assets/                      | Settings → Files and links            |
| 권장(단축키)  | Download attachments for current file                     | Settings → Hotkeys                    |
| 선택(커뮤니티) | Dataview                                                  | Settings → Community plugins          |
| 선택(커뮤니티) | Marp                                                      | Settings → Community plugins          |
| 선택(확장)   | Obsidian Web Clipper                                      | 브라우저 확장 프로그램 설치                       |
| 선택(고급)   | Karpathy LLM Wiki 플러그인 / 로컬 검색(qmd)                       | Settings → Community plugins / CLI 설치 |

핵심만 요약하면, **코어 플러그인 5개(백링크·그래프뷰·아웃고잉링크·검색·프로퍼티스)만 켜져 있으면 카파시가 설명한 최소 요건은 충족**됩니다. Dataview와 Marp는 위키가 성숙해지면서 표나 슬라이드 같은 부가 산출물이 필요할 때 추가하시면 됩니다.

Q4. 사용자

Dataview와 Marp를 처음부터 추가해도 상관은 없지?

A4. 어시스턴트(Claude)

네, 처음부터 추가하셔도 전혀 문제없습니다.

- **Dataview 와 Marp 는 코어 플러그인이 아니라 완전히 독립적인 커뮤니티 플러그인**이라서, 다른 기능들과 충돌하거나 위키 구조 자체에 영향을 주지

않습니다. 커놓아도 아무 일도 안 하다가 필요할 때(YAML 프론트매터를 채우거나, 슬라이드를 만들 때) 쓰면 됩니다.

- 오히려 나중에 추가하려고 하면 "이 페이지들에 프론트매터가 없어서 Dataview 쿼리가 안 돌아간다"는 식으로 소급 작업이 필요해질 수 있는데, 처음부터 CLAUDE.md 스키마에 "모든 페이지에 YAML 프론트매터로 tags, date, source\_count 등을 포함하라"고 규칙을 넣어두면 나중에 Dataview 를 쓸 때 훨씬 매끄럽습니다.
- 리소스 부담도 거의 없습니다 — 둘 다 가벼운 렌더링/쿼리 플러그인이라 볼트 성능에 영향을 주지 않습니다.

정리하면, **미리 커두고 스키마에 관련 규칙만 반영해두는 것이 오히려 더 유리한 선택**입니다. 순서 걱정 없이 처음부터 함께 설치하시면 됩니다.

## Q5. 사용자

---

안드레이 카파시의 "LLM Wiki" 개념

LLM Wiki는 카파시가 2026년 4월 4일 GitHub Gist(llm-wiki.md)로 공개한 아이디어로, 완성된 제품이나 코드가 아니라 기존 RAG 방식처럼 매번 원본 문서에서 정보를 재검색하는 대신, LLM이 큐레이션된 소스 자료를 점진적으로 지속적이고 상호연결된 마크다운 위키로 컴파일하도록 하는 "아이디어 파일"입니다.

핵심 개념

대부분의 RAG 방식(NotebookLM, ChatGPT 파일 업로드 등)은 문서를 업로드하면 질의 시점에 관련 조각을 검색해서 답을 생성하는데, 이 경우 LLM은 매 질문마다 지식을 처음부터 재발견하게 되어 축적이 일어나지 않습니다. LLM Wiki는 다릅니다. 새 소스를 추가하면 LLM이 단순히 색인만 하는 게 아니라 내용을 읽고 핵심 정보를 추출해 기존 위키에 통합합니다 — 엔티티 페이지를 업데이트하고, 주제 요약을 수정하고, 새 데이터가 기존 주장과 모순되는 부분을 표시하고, 종합(synthesis)을 강화하거나 도전합니다. 즉 지식이 한 번 컴파일되고 계속 최신 상태로 유지되는 것이지, 매 질의마다 다시 도출되는 게 아닙니다.

니가 설명한 LLM Wiki에 대한 위 설명이 너무 어렵고 문장의 단어나 내용도 어색해.

1. Gist 가 뭐야?
2. 큐레이션된 소스 자료 라는 내용에서 큐레이션 되었다는게 무슨 뜻이야?



3. 마크다운 위키로 컴파일하도록 이라는 내용에서 마크다운 위키가 뭐고 컴파일 한다는것도 뭐야?
4. 엔티티 페이지 는 뭐야?
5. 종합(synthesis)을 강화하거나 도전 한다는게 굉장히 어색한데 종합은 뭐고 강화하거나 도전한다는게 무슨 말이야?

## A5. 어시스턴트(Claude)

---

설명이 너무 어려웠네요. 하나씩 쉽게 풀어드리겠습니다.

### 1. Gist 가 뭐야?

**Gist**는 GitHub(개발자들이 코드를 공유하는 사이트)에서 제공하는 기능으로, 짧은 글이나 코드 조각을 간단히 올려서 다른 사람과 공유할 수 있는 작은 게시글입니다. 블로그 글을 쓰기엔 부담스럽고, 그냥 메모장처럼 빠르게 아이디어를 적어서 링크 하나로 공유하고 싶을 때 씁니다. 카파시는 정식 논문이나 블로그 포스트가 아니라 이 Gist 형태로 "LLM Wiki"라는 아이디어를 가볍게 정리해서 올린 것입니다.

### 2. "큐레이션된 소스 자료"에서 큐레이션이 무슨 뜻이야?

**큐레이션(curation)**은 원래 미술관에서 큐레이터가 전시할 작품을 골라서 정리하는 것에서 온 말입니다. 여기서는 그냥 "**아무 자료나 다 넣는 게 아니라, 내가 직접 고르고 선별한 자료**"라는 뜻입니다.

예를 들어 인터넷에 있는 모든 문서를 무작위로 넣는 게 아니라, "이 논문은 중요하니까 넣자", "이 기사는 도움이 될 것 같으니 넣자" 하고 사람이 직접 골라서 모아놓은 자료라는 의미입니다.

### 3. "마크다운 위키로 컴파일"에서 마크다운 위키가 뭐고, 컴파일은 뭐야?

**마크다운(Markdown)**은 텍스트 파일을 꾸미는 아주 간단한 문법입니다. 예를 들어 # 제목이라고 쓰면 큰 제목이 되고, **\*\*굵게\*\***라고 쓰면 굵은 글씨가 됩니다. 워드처럼 복잡한 프로그램 없이, 그냥 메모장에 글자 몇 개만 더해서 문서 형식을 만드는 방식이라고 생각하시면 됩니다. 확장자는 **.md**입니다.

**위키(Wiki)**는 위키백과(나무위키 같은 것)처럼, 여러 페이지가 서로 링크로 연결되어 있는 문서 모음입니다. "인공지능" 페이지를 보다가 "머신러닝"이라는 단어를 클릭하면 그 페이지로 넘어가는 식입니다.

즉 **"마크다운 위키"**는 이런 위키백과 같은 문서들을, 워드나 전용 프로그램이 아니라 그냥 마크다운이라는 간단한 텍스트 형식으로 만든 것을 말합니다.

**컴파일(compile)**은 원래 프로그래밍 용어로, 사람이 쓴 코드를 컴퓨터가 이해할 수 있는 형태로 "변환하고 정리하는" 작업을 뜻합니다. 여기서는 **"흩어져 있는 원본 자료들을 읽고, 정리해서, 하나의 체계적인 위키 문서로 만들어내는 것"**을 비유적으로 표현한 것입니다. 즉 논문 10개를 던져주면 LLM이 그걸 다 읽고 정리된 위키 페이지들로 "만들어낸다"는 뜻입니다.

#### 4. 엔티티 페이지가 뭐야?

**엔티티(entity)**는 "구체적인 대상"이라는 뜻입니다. 사람 이름, 회사 이름, 제품 이름, 개념 이름처럼 하나의 독립된 대상을 가리킵니다.

**엔티티 페이지**는 그 대상 하나마다 만들어진 전용 페이지입니다. 예를 들어 위키백과에서 "삼성전자"를 검색하면 삼성전자에 대한 페이지가 따로 있는 것처럼, LLM Wiki에서도 "테슬라", "일론 머스크", "리튬 배터리" 같은 개별 대상마다 각각 페이지를 하나씩 만들어서 관리합니다. 새로운 자료를 넣을 때마다 그 자료에 언급된 대상의 페이지가 자동으로 업데이트됩니다.

#### 5. "종합을 강화하거나 도전한다"는 게 무슨 말이야?

**종합(synthesis)**은 여러 자료를 읽고 나서 "결국 이게 무슨 의미인가"를 하나로 정리한 나만의 결론이나 분석을 말합니다. 예를 들어 논문 5개를 읽고 "결론적으로 A 이론이 B 이론보다 설득력 있다"라고 정리했다면, 그게 종합입니다.

- **강화한다**는 것은, 새로 넣은 자료가 기존 결론과 **같은 방향을 뒷받침해줄 때** — "역시 맞았네, 근거가 하나 더 늘었다"는 뜻입니다.
- **도전한다**는 것은, 새로 넣은 자료가 기존 결론과 **반대되는 내용일 때** — "어? 이 자료는 기존 결론이랑 안 맞는데?"라고 모순을 알려준다는 뜻입니다.

즉 새 자료를 넣을 때마다 LLM이 "이 자료가 기존에 내가 내린 결론을 더 확실하게 만

들어주는지, 아니면 흔들거나 반박하는지"를 판단해서 위키에 반영해준다는 의미입니다.

## Q6. 사용자

---

3단계 아키텍처에서 The Wiki 설명에서 LLM이 생성한 마크다운 파일들 이라는게 뭐야 Raw sources로 부터 뭔가 생성한거야? 그럼 뭘 기준으로 생성을 하는거야? 사용자가 뭐라고 요청을 했으니 생성했을건데?

3단계 아키텍처 들의 3가지 중 원본 소스 말고는 언제 누구에 의해서 어떤 요청으로 인해서 언제 어떻게 생성되는건지 모르겠어

## A6. 어시스턴트(Claude)

---

좋은 질문이에요. 순서를 시간 흐름으로 다시 짚어드리면 훨씬 명확해집니다.

결론부터: 맞습니다, Raw sources 를 기준으로 생성하는 거예요

**위키(wiki) 페이지**는 사용자가 "이 자료를 넣어줘(ingest)"라고 요청하면, 그 순간 LLM 이 raw sources 폴더에 있는 원본 문서를 읽고, 그 내용을 근거로 만들어내는 겁니다. 아무 근거 없이 LLM이 알아서 상상해서 쓰는 게 아니라, 반드시 원본 문서 내용을 기반으로 생성됩니다.

시간 순서로 다시 설명드리면

### 1단계 — 최초 설정 (사람이 딱 한 번 함)

사용자가 Claude Code 같은 LLM 에이전트에게 "이런 폴더 구조로 위키를 만들고 싶어. 스키마 파일을 만들어줘"라고 요청합니다.

→ 이때 **스키마(CLAUDE.md)**가 생성됩니다. 이건 "앞으로 위키를 이렇게 관리해라"는 규칙 문서입니다. 이 시점엔 아직 raw sources도 wiki도 비어있습니다.

### 2단계 — 원본 소스 추가 (사람이 매번 함)

사용자가 논문 PDF나 기사를 raw/ 폴더에 넣습니다.

→ 이건 단순히 파일을 복사해 넣는 것뿐, 아직 아무것도 생성되지 않습니다.

### 3단계 — Ingest 요청 (사람이 명시적으로 요청)

사용자가 LLM 에이전트에게 "방금 넣은 이 논문 수집(ingest)해줘"라고 지시합니다.

→ 바로 이 순간 LLM이:

6. raw/ 폴더의 그 문서를 읽고
7. 스키마(CLAUDE.md)에 적힌 규칙을 따라
8. 그 문서 내용을 근거로 wiki/ 폴더 안에 마크다운 페이지들을 새로 쓰거나(생성), 이미 있던 페이지가 있으면 고쳐씁니다(수정)

예를 들어 "리튬 배터리 열화" 논문을 ingest하면, LLM은 그 논문을 읽고 wiki/리튬배터리.md라는 페이지를 새로 만들거나(처음이면), 이미 있으면 그 내용을 업데이트합니다. 그리고 논문에 "테슬라"가 언급됐으면 wiki/테슬라.md 페이지도 같이 업데이트합니다.

즉 정리하면

| 계층                 | 누가 만드나 | 언제 만들어지나                          | 무엇을 근거로                         |
|--------------------|--------|-----------------------------------|---------------------------------|
| Raw sources        | 사람     | 사람이 파일을 폴더에 넣을 때                  | (원본 그 자체, 생성 아님)                |
| Schema (CLAUDE.md) | LLM    | 최초 설정 시, 사람이 "규칙 파일 만들어줘"라고 요청할 때 | 사람과의 대화(어떤 종류 위키인지, 어떤 규칙 원하는지) |
| Wiki 페이지들          | LLM    | 사람이 "이 소스 ingest해줘"라고 요청할 때마다     | raw sources 안의 원본 문서 내용         |

그러니까 처음 걱정하신 것처럼 "LLM이 뭘 기준으로 마음대로 생성하나"가 아니라, "사람이 원본 자료를 넣고 → 사람이 처리하라고 명령하면 → 그 원본 자료 내용을 근거로 LLM이 위키 페이지를 만든다"는 순서입니다. 사람의 개입(자료 큐레이션 + ingest 명령)이 매 단계마다 들어갑니다.

## Q7. 사용자

사용자가 Claude Code 같은 LLM 에이전트에게 "이런 폴더 구조로 위키를 만들고 싶어. 스키마 파일을 만들어줘"라고 요청합니다. 라는 내용에서 이런 폴더 구조로 위키를 만

들고 싶어 라고 할때 폴더 구조를 미리 어딘가에 적어놔야 하는거야? 그냥 이런 폴더 구조 라는 말로만으로도 뭘 어떻게 만들지 모를거 같은데?

## A7. 어시스턴트(Claude)

아, 제가 표현을 애매하게 해서 오해가 생겼네요. **미리 폴더 구조를 문서로 적어둘 필요는 없습니다.** 실제로는 이렇게 진행됩니다.

실제로는 이렇게 흘러갑니다

**1. 사용자가 미리 준비하는 건 폴더 구조가 아니라 "카파시의 원본 아이디어 글"입니다.**

이전에 제가 정리해드린 카파시 Gist 내용(핵심 개념, 3단계 아키텍처, ingest/query/lint 운영 방식)을 그대로 복사해서 Claude Code에게 붙여넣습니다. 즉 "폴더 구조를 내가 설계해서 알려주는 것"이 아니라 **"이런 패턴의 위키를 만들고 싶다"**는 **개념 설명을 던져주는 것**입니다.

**2. Claude Code가 먼저 질문을 합니다.**

아이디어 글만 읽고 Claude Code가 바로 폴더를 막 만드는 게 아니라, 보통 이런 식으로 되물어옵니다:

> "어떤 주제의 위키를 만드실 건가요? 어떤 종류의 자료(논문, 기사, 이미지 등)를 넣으실 계획인가요?"

**3. 사용자가 자기 상황을 답합니다.**

예: "GD RDM 프로젝트 관련 갤럭시 진단 CSV 스키마 자료들을 모아서 위키로 만들고 싶어."

**4. 그 답변을 바탕으로 Claude Code가 폴더 구조를 스스로 제안하고 만듭니다.**

예를 들면 이런 식으로 (실제로는 사용자 답변에 맞춰 조금씩 달라짐):

```
my-wiki/
├── raw/           ← 원본 CSV 스펙 문서, 회의록 등
├── wiki/
│   ├── index.md
│   └── log.md
└── CLAUDE.md     ← 방금 대화 내용을 바탕으로 LLM이 직접 작성
```

즉 폴더 구조를 "설계"하는 주체는 사용자가 아니라 LLM 에이전트입니다. 사용자는 "

이런 게 필요해"라는 목적과 맥락만 말로 설명하면 되고, 실제 디렉터리 이름·구조·CLAUDE.md 규칙 내용은 Claude Code가 카파시의 패턴을 참고해서 알아서 초안을 짭니다.

### 그럼 사용자가 할 일은 뭐야?

정리하면 사용자가 사전에 준비할 건 딱 두 가지뿐입니다.

9. 카파시의 아이디어 글(제가 정리해드린 내용) — Claude Code에게 "이 패턴대로 만들어줘"라고 보여줄 참고자료
10. "나는 이런 주제/목적으로 위키를 쓰고 싶다"는 대략적인 설명 — Claude Code가 물어보면 답할 내용

폴더 이름이나 몇 단계로 나눌지 같은 구체적인 설계는 전혀 미리 고민하실 필요 없고, 대화하면서 Claude Code가 제안한 걸 보고 "이건 이렇게 바꿔줘"라고 조정하시면 됩니다.

## Q8. 사용자

---

1단계에서는 그럼 카파시의 LLM wiki 문서만 주는거네? 그리고 나서 내가 원하는 주제를 주면 Claude code가 폴더 구조를 작성하고 그 다음부터 2단계로 넘어가 내가 주제와 관련된 파일을 주다음 3단계에서 수집해 줘라고 하는거야? 그럼 이때 2단계에서 한번에 모든 주제 관련된 파일을 다 못주니까 예를 들어 10개만 먼저 주고 수집해 달라고 한 후 다음에 또 자료가 구해지면 또 2단계 처럼 자료를 넘겨주고 그 다음에 3단계 처럼 수집해달라고 하는 패턴들이 반복되는거야?

## A8. 어시스턴트(Claude)

---

네, 정확하게 이해하셨습니다. 정리하신 흐름이 맞고, 딱 하나만 보충해서 설명드리면 완벽합니다.

### 이해하신 흐름 확인

11. **1 단계 (최초 1 회만):** 카파시 문서를 Claude Code에게 던져주고, 원하는 주제를 알려주면 → Claude Code가 폴더 구조(raw/, wiki/, CLAUDE.md)를 만들어줌
12. **2 단계:** 자료 10 개 정도를 raw/ 폴더에 넣음

13. **3 단계:** "방금 넣은 10 개 수집(ingest)해줘"라고 요청 → 위키 페이지들 생성/업데이트
14. 나중에 자료가 더 생기면 → **2 단계, 3 단계를 다시 반복** (자료 넣기 → ingest 요청)

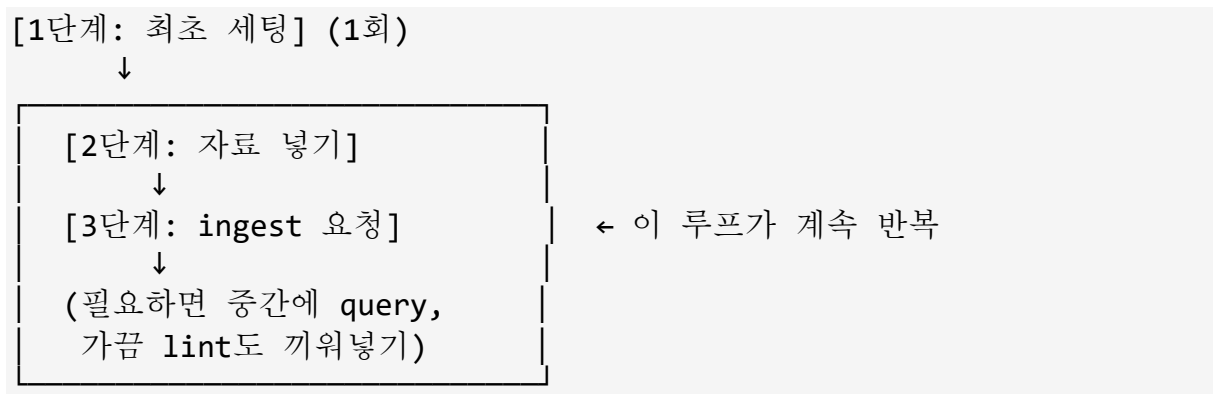
이게 맞습니다. 1단계는 딱 한 번뿐이고, 2~3단계는 자료가 생길 때마다 계속 반복되는 **루프(loop)**입니다.

한 가지 보충 — 반복되는 건 2~3 단계뿐만이 아니에요

실제로는 이 루프에 두 가지가 더 끼어듭니다:

- **Query(질의):** 자료를 쌓아가는 중간중간 "지금까지 모은 자료 기준으로 A와 B를 비교해줘" 같은 질문을 던질 수 있습니다. 이건 새 자료를 넣는 게 아니라 이미 쌓인 위키에 질문만 하는 거라, 언제든지 아무 때나 할 수 있습니다.
- **Lint(점검):** 자료가 어느 정도 쌓이면(예: 몇 주에 한 번 정도) "위키 전체 점검해줘"라고 요청해서, 모순되는 내용이나 연결 안 된 고아 페이지가 있는지 확인받을 수 있습니다.

그래서 전체 사이클은 이런 모양이 됩니다



즉 사용자가 이해하신 "10개 넣고 수집 → 나중에 또 넣고 또 수집" 패턴이 이 시스템의 기본 골격이고, 거기에 필요할 때마다 질문(query)과 가끔 점검(lint)이 추가되는 구조라고 보시면 됩니다.

## Q9. 사용자

소스는 어떤 포맷들을 지원해? 지원되는 모든 파일의 형식을 알려줘

## A9. 어시스턴트(Claude)

### LLM Wiki 의 raw sources 가 지원하는 파일 형식

카파시의 원본 아이디어 자체는 특정 포맷을 강제하지 않습니다. "원본 소스는 논문, 기사, 이미지, 데이터 파일 등 큐레이션된 소스 문서 모음"이라고만 되어 있고, 실제로 어떤 형식까지 읽을 수 있는지는 **사용하는 LLM 에이전트(Claude Code)가 무엇을 읽을 수 있는지에 달려 있습니다.** Claude Code 기준으로 정리해드리면 다음과 같습니다.

### 텍스트 계열 (가장 안정적으로 지원)

| 형식                 | 설명                                          |
|--------------------|---------------------------------------------|
| .md (마크다운)         | 가장 이상적인 포맷. Web Clipper로 저장한 아티클이 대부분 이 형식  |
| .txt               | 순수 텍스트                                      |
| .csv, .tsv         | 표 형태 데이터. GD RDM의 진단 CSV 스키마 자료 같은 것도 여기 해당 |
| .json, .yaml       | 구조화된 데이터                                    |
| 코드 파일 (.py, .js 등) | 코드베이스나 스크립트도 소스로 넣을 수 있음                    |

### 문서 계열

| 형식            | 지원 여부                                                       |
|---------------|-------------------------------------------------------------|
| .pdf          | 지원됩니다. 텍스트 추출은 물론, 스캔본이라도 페이지를 이미지로 변환해서 시각적으로 읽는 것까지 가능합니다 |
| .docx (워드)    | 지원됩니다. Skill 기능으로 내용을 읽을 수 있습니다                             |
| .pptx (파워포인트) | 지원됩니다. 슬라이드별 텍스트와 이미지를 함께 읽을 수 있습니다                         |
| .xlsx (엑셀)    | 지원됩니다. 시트/셀 데이터를 읽을 수 있습니다                                  |
| .html         | 지원됩니다. 웹페이지를 저장한 형태                                         |



이미지 계열 (비전 기능 필요)

| 형식                                                                                   | 설명                                                                                           |
|--------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| <code>.png</code> , <code>.jpg/.jpeg</code> , <code>.gif</code> , <code>.webp</code> | Claude가 시각적으로 직접 보고 해석 가능. 카파시도 이미지를 로컬로 다운로드해 <code>raw/assets/</code> 에 저장하는 것을 팁으로 언급했습니다 |
| 스크린샷, 화이트보드 사진, 슬라이드 캡처, 다이어그램                                                       | 텍스트와 동일한 방식으로 ingest 가능하며, 비전 지원 모델이 이미지 속 텍스트는 그대로 옮기고 해석이 필요한 부분은 별도로 표시할 수 있습니다           |

오디오/영상 계열 — 직접 지원은 안 됨, 변환 필요

| 형식                  | 처리 방법                                                                                                                                                                                |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 팟캐스트, 유튜브 영상, 녹음 파일 | Claude가 오디오/영상 자체를 직접 듣거나 볼 수는 없습니다. 별도 도구 (Whisper 같은 음성인식 도구, 유튜브 자막 추출 등)로 <b>먼저 텍스트 스크립트로 변환한 뒤</b> <code>raw/</code> 에 넣어야 합니다. 카파시도 "팟캐스트 노트"라고 표현했지 팟캐스트 음성 파일 자체를 언급하진 않았습니다 |

압축/아카이브

| 형식                  | 처리 방법                                                                                                 |
|---------------------|-------------------------------------------------------------------------------------------------------|
| <code>.zip</code> 등 | Claude Code가 bash 명령으로 압축을 풀어서 안의 개별 파일들을 각각 읽을 수 있습니다. 압축 파일 자체를 소스로 넣기보다는 풀어서 개별 파일 단위로 정리하는 게 낫습니다 |

정리 — 실질적인 기준

"글자로 읽히거나, 눈으로 보이는 것"이면 대부분 raw sources로 쓸 수 있습니다.

- 글자: md, txt, pdf, docx, pptx, xlsx, csv, json, html, 코드 파일
- 이미지: png, jpg, gif, webp, 스캔본 PDF
- 예외(변환 필요): 오디오·영상은 텍스트로 먼저 변환해야 함

GD RDM 프로젝트 맥락이면 CSV 스펙 문서, 관련 PDF 스펙, 회의록 텍스트, 스크린샷 (진단 화면 캡처 등) 정도가 자연스러운 raw sources 조합이 될 것 같습니다.

## Q10. 사용자

---

그럼 전체적으로 사람이 개입되는 부분이 어떤것들이야?

## A10. 어시스턴트(Claude)

---

LLM Wiki 전체 흐름에서 사람이 개입하는 지점 정리

지금까지 나눈 대화를 시간 순서로 모아서, 사람이 직접 손을 대는 부분만 뽑아드리겠습니다.

### 1. 최초 설정 단계

- **아이디어 문서 전달**: 카파시의 Gist 내용을 Claude Code 에게 붙여넣는 것
- **주제/목적 설명**: "어떤 위키를 만들고 싶은지" 질문에 답하기 (예: "GD RDM 프로젝트 진단 CSV 스키마 위키")
- **폴더 구조 검토·조정**: Claude Code 가 제안한 구조를 보고 "이건 이렇게 바꿔줘"라고 조정
- **스키마(CLAUDE.md) 검토**: LLM 이 작성한 규칙 문서가 마음에 드는지 확인하고 수정 요청

### 2. 자료 수집(Ingest) 단계 — 반복되는 핵심 개입

- **소스 큐레이션(가장 중요)**: 어떤 자료를 넣을지, 어떤 자료는 뺄지 사람이 직접 판단해서 고름. LLM 은 넣어준 것만 처리하지, 스스로 자료를 찾아 넣지 않음

- **ingest 명령:** "이 자료 수집해줘"라고 매번 직접 요청해야 함 — 파일을 raw/에 넣어놔도 자동으로 처리되지 않음
- **요약 검토 및 방향 지시:** 소스를 하나씩 수집하며 계속 관여하는 방식을 선호한다고 언급했듯, 생성된 요약물을 읽고 "이 부분을 더 강조해줘" 같은 피드백을 줄 수 있음 (물론 감독을 덜 하고 여러 개를 한꺼번에 일괄 수집시키는 것도 가능 — 이것도 사람의 선택)

### 3. 질의(Query) 단계

- **질문 던지기:** 위키에 뭘 물어볼지는 전적으로 사람의 몫. LLM 이 알아서 질문을 만들어 스스로 답하지 않음
- **좋은 답변 파일링 결정:** 답변이 유용하면 "이것도 위키 페이지로 저장해줘"라고 지시해야 대화 기록에서 사라지지 않고 축적됨

### 4. 점검(Lint) 단계

- **린트 실행 요청:** "위키 점검해줘"라고 주기적으로 직접 요청해야 함 — 자동으로 돌아가지 않음
- **최종 판단:** LLM 이 "이 두 페이지가 모순됩니다", "이 페이지는 고아입니다"라고 보고하면, 실제로 무엇을 고칠지, 어떤 링크가 의미 있는지, 어떤 페이지를 만들지, 어떤 제안은 아직 시기상조인지, 어떤 이슈는 그냥 둘지를 **최종적으로 결정하는 건 사람임** (LLM 은 편집자가 아니라 제안자)

### 5. Obsidian 을 통한 상시 관찰

- **그래프 뷰·백링크 확인:** LLM 이 편집하는 동안 실시간으로 그래프를 따라가며 결과를 지켜보는 것 — 카파시 표현으로는 "LLM 에이전트를 한쪽에, Obsidian 을 다른 쪽에 열어두고" 작업하는 방식. 사람은 항상 읽기 모드에 있고 절대 위키 파일을 직접 편집하지 않지만, **눈으로 계속 지켜보고 이상하면 다음 요청에서 방향을 바로잡음**

### 한 줄 요약

**LLM이 하는 일:** 읽기, 요약, 상호참조, 부기(bookkeeping), 일관성 유지 — 즉 "지루하고

반복적인 노동"

**사람이 하는 일:** 무엇을 넣을지 고르기(큐레이션), 언제 처리할지 명령하기 (ingest/query/lint 트리거), 결과가 맞는지 판단하고 방향 잡기(편집·최종 결정)

한마디로 "LLM이 손발을 움직이고, 사람이 머리와 눈 역할을 계속 담당한다"는 구조입니다. 완전 자동화가 아니라, 매 단계마다 사람의 명령과 판단이 필요한 반(半)자동 시스템입니다.

## Q11. 사용자

---

그럼 내가 원하는 주제별로 각각 별도의 LLM Wiki를 만드는거야?

## A11. 어시스턴트(Claude)

---

네, 기본적으로는 **주제(또는 목적)별로 별도의 위키(=별도의 폴더/Obsidian 볼트)**를 만드는 것이 일반적입니다. 이유와 함께 정리해드리겠습니다.

왜 주제별로 분리하는 게 자연스러운가

카파시가 예로 든 활용 사례들 — 개인 목표/건강 추적, 연구 주제 딥다이브, 책 읽기, 비즈니스/팀 내부 위키, 경쟁사 분석, 여행 계획, 강의 노트, 취미 딥다이브 — 은 각각 성격이 완전히 다릅니다. 이걸 하나의 위키에 다 섞으면:

- **스키마(CLAUDE.md) 규칙이 충돌:** "GD RDM 위키"는 CSV 스키마 필드, 검증 규칙 위주로 페이지를 짜야 하는데, "KOSPI 모니터링 위키"는 종목, 지표, 알림 로직 위주로 짜야 함. 하나의 규칙 파일로 둘 다 만족시키기 어려움
- **그래프가 지저분해짐:** Obsidian 그래프 뷰에서 서로 무관한 두 주제의 노드가 뒤섞이면 "어떤 개념이 진짜 중요한 허브인지" 파악이 안 됨
- **인덱스(index.md)가 뒤섞임:** 질의할 때 LLM 이 index.md 를 먼저 읽고 관련 페이지를 찾는데, 무관한 주제 페이지들이 섞여 있으면 탐색 효율이 떨어짐

그래서 실제로는 이렇게 나누는 게 일반적

최상길님 상황에 대입하면 이런 식이 될 것 같습니다:

```
~/claude/projects/
├── gd-rdm-wiki/           ← GD RDM 프로젝트 전용
│   ├── raw/
│   ├── wiki/
│   └── CLAUDE.md
├── kospi-wiki/           ← KOSPI 모니터링 전용
│   ├── raw/
│   ├── wiki/
│   └── CLAUDE.md
└── a7m4-wiki/            ← A7M4 가이드 앱 자료 전용
    ├── raw/
    ├── wiki/
    └── CLAUDE.md
```

각 폴더가 독립된 Obsidian 볼트가 되고, 각각 별도의 스키마·인덱스·로그를 가집니다.

### 예외 — 합쳐도 되는 경우

다만 무조건 나눠야 하는 규칙은 아닙니다. 예를 들어:

- 주제들이 서로 강하게 연결되어 있고 (예: "개인 성장"이라는 큰 우산 아래 건강·독서·목표 추적이 다 얹혀 있는 경우), 상호참조가 오히려 가치를 만든다면 하나의 위키로 합치는 것도 카파시가 예시로 든 "개인" 카테고리처럼 자연스러운 선택입니다.
- 반대로 GD RDM 처럼 업무적으로 완전히 독립된 프로젝트는 분리하는 편이 관리도 쉽고 그래프도 깔끔합니다.

기준은 "이 두 주제의 페이지들이 서로 링크될 일이 실제로 있는가"입니다. 링크될 일이 거의 없다면 나누고, 서로 참조하며 시너지가 나는 관계라면 합치는 게 낫습니다.

## Q12. 사용자

LLM Wiki 내에서 문서들 간의 link를 설정한다고 하던데 이걸 Claude 가 알아서 하는거야?

### A12. 어시스턴트(Claude)

네, 맞습니다. 링크 연결은 사람이 수동으로 하는 게 아니라 Claude(LLM)가 알아서 합니다.

## 왜 그런가

3단계 아키텍처에서 이미 설명드렸듯이, 위키 계층은 LLM이 전적으로 소유(own)합니다. "사용자는 읽고, LLM은 쓴다"는 원칙이 여기에도 그대로 적용됩니다. 링크도 위키 페이지의 일부이기 때문에 LLM이 만드는 대상에 포함됩니다.

## 실제로 어떻게 일어나는가

**ingest(수집) 시점**에 자동으로 이루어집니다. 예를 들어 "테슬라 배터리 열화" 논문을 ingest하면:

15. LLM 이 논문을 읽으면서 "리튬 배터리", "테슬라", "배터리 관리 시스템" 같은 개념/엔티티가 언급된 걸 파악
16. 이미 위키에 `wiki/리튬배터리.md` 페이지가 있으면 → 새 논문 내용을 반영해 업데이트하면서, 새로 만드는 `wiki/테슬라배터리열화.md` 페이지에서 `[[리튬배터리]]` 형식으로 링크를 겁니다
17. 반대로 `리튬배터리.md` 페이지에도 "이 새 논문에서도 다룬다"는 식으로 역방향 언급이 추가됨 (이게 Obsidian 의 백링크로 자동 표시됨)

즉 사용자는 `[[ ]]` 문법을 직접 타이핑할 필요가 전혀 없고, Claude Code가 문서를 읽으면서 스스로 "이 단어는 이미 위키에 있는 개념이니까 링크해야겠다"고 판단해서 위키링크를 심습니다.

## 사람이 개입하는 부분은?

- **스키마(CLAUDE.md)에 링크 규칙을 정해두는 것**: 예를 들어 "새로운 개념을 언급할 때는 반드시 `[[개념명]]`으로 링크하라", "이미 존재하는 페이지 이름과 비슷하면 새로 만들지 말고 기존 페이지에 병합하라" 같은 규칙을 최초 설정 시 정해둘 수 있습니다.
- **Lint 시점의 최종 판단**: LLM 이 "이 두 페이지는 비슷한 개념 같은데 하나로 합칠까요?", "이 페이지는 링크가 하나도 없는 고아 페이지입니다"라고 보고하면, 실제로 합칠지·새 링크를 만들지는 사람이 결정합니다. LLM 이 제안한 별칭(alias)만으로 자동 병합하는 건 신뢰할 근거가 약하다고 보고, 사람의 최종 검토를 거치는 방식을 권장하는 구현도 있습니다.

정리하면, **평상시 링크 생성은 100% LLM이 자동으로 처리**하고, 사람은 최초에 규칙만 정해두거나 나중에 애매한 병합 판단만 개입하는 구조입니다.

### Q13. 사용자

---

그럼 Obsidian의 역할은 이런 구조들을 화면으로 보여주는거야? 정확히 LLM Wiki에서 Obsidian의 역할/기능은 뭐야? 애가 뭔가 이런 구조나 파일들을 편집할 수도 있게 해주는거야? 예를 들어 위에서 링크를 설정해 줬지만 내가 보기엔 적절한 링크가 아니라면 수정이 필요할 수도 있잖아.

### A13. 어시스턴트(Claude)

---

좋은 질문입니다. 이 부분이 좀 헷갈리기 쉬운데, **"권장하는 사용 방식"**과 **"기술적으로 가능한 것"**을 나눠서 설명드리겠습니다.

#### Obsidian의 정체 — 사실은 그냥 "마크다운 편집기"입니다

Obsidian은 원래 마크다운 파일(.md)을 읽고 쓰는 범용 텍스트 에디터입니다. 위키/그래프뷰/백링크 같은 기능이 얹혀 있는 거지, 근본적으로는 메모장 같은 편집 프로그램입니다. 즉 **기술적으로는 Obsidian 안에서 위키 페이지를 직접 열어서 사람이 타이핑으로 수정하는 것도 100% 가능합니다**. 파일이 그냥 디스크 위의 평범한 텍스트 파일이기 때문입니다.

#### 그런데 왜 "읽기 전용으로 쓰라"고 권장하는가

카파시가 설명한 워크플로우는 사람이 항상 읽기 모드에 있고 편집 모드에는 있지 않다 — 즉 브라우징하고, 링크를 따라가고, 그래프를 확인하고, 읽기만 하며 위키 파일을 직접 편집하지 않는 것입니다. 이건 **기술적 제약이 아니라 워크플로우 철학**입니다. 이유는:

- 사람이 직접 고쳐버리면, 그 페이지가 왜 이렇게 됐는지에 대한 "생성 히스토리"가 끊깁니다. LLM이 다음번에 그 페이지를 다시 건드릴 때 사람이 손으로 고친 부분과 자기가 쓴 부분을 구분 못 하고 헷갈릴 수 있습니다.

- 사람이 하나씩 손으로 고치기 시작하면, 애초에 이 패턴을 쓰는 이유였던 "부기(bookkeeping)를 LLM에게 맡긴다"는 장점이 사라집니다.

그래서 질문하신 "링크가 이상할 때"는 어떻게 하나요?

**직접 Obsidian에서 타이핑으로 고치는 게 아니라, Claude Code에게 말로 지시하는 게 권장 방식입니다.**

예: "테슬라배터리열화.md에서 [[리튬배터리]] 링크가 잘못됐어. 사실 이건 [[전고체배터리]]를 가리켜야 해. 고쳐줘."

→ Claude Code가 그 파일을 다시 열어서 링크를 수정하고, 필요하면 관련된 다른 페이지(백링크가 걸린 리튬배터리.md, 전고체배터리.md)까지 일관성 있게 함께 손봅니다. 사람이 직접 고치면 놓칠 수 있는 "연쇄 수정"을 LLM이 처리해주는 게 핵심 가치입니다.

정리 — Obsidian의 진짜 역할 3 가지

| 역할                    | 설명                                                                                    |
|-----------------------|---------------------------------------------------------------------------------------|
| 1. 뷰어/탐색기             | 그래프 뷰, 백링크, 검색으로 위키 구조를 시각적으로 파악. Claude Code가 파일을 수정하면 실시간으로 반영되어 보임                 |
| 2. 편집 가능한 에디터 (기술적으로) | 마크다운 파일이므로 직접 타이핑 수정도 가능한 함 — 다만 이건 워크플로우 원칙상 권장되지 않음                                 |
| 3. 문제 발견 도구           | 그래프에서 이상한 연결이나 고아 페이지를 사람이 눈으로 발견하면, 그걸 다시 Claude Code에게 "이거 고쳐줘"라고 되돌려주는 피드백 루프의 시작점 |

즉 Obsidian이 "고칠 수 있게 해주느냐"는 질문에 답하면 — **기술적으로는 가능하지만, 이 패턴의 정신은 "눈으로 문제를 발견 → 말로 LLM에게 수정 요청"이지, "직접 손으로 고치기"가 아닙니다.** 직접 고치고 싶으시면 언제든지 하실 수 있지만, 그러면 일반 Obsidian 볼트를 쓰는 것과 다를 게 없어지고 이 패턴의 장점(LLM이 연쇄적으로 일관성을 유지해주는 것)을 포기하는 셈이 됩니다.

## Q14. 사용자

일반 Obsidian 볼트 라는건 뭐야?



## A14. 어시스턴트(Claude)

**Obsidian 볼트(vault)**는 그냥 "Obsidian이 관리하는 폴더 하나"를 부르는 말입니다. 특별한 게 아니라, Obsidian에서 "Open folder as vault"로 연 폴더 자체를 볼트라고 부릅니다.

### "일반 Obsidian 볼트"라고 한 이유

앞서 설명드린 LLM Wiki도 사실 기술적으로는 그냥 "마크다운 파일들이 든 폴더"이고, 그걸 Obsidian으로 열면 볼트가 됩니다. 그러니 **LLM Wiki의 wiki/ 폴더도 결국 하나의 Obsidian 볼트**입니다.

제가 "일반 Obsidian 볼트를 쓰는 것과 다를 게 없어진다"고 한 말은 이런 뜻입니다:

- **원래 Obsidian은 원래 사람이 직접 손으로 메모하고 링크를 걸면서 쓰는 개인 노트 프로그램**입니다. 수많은 사람들이 이렇게 씁니다: 매일 일기를 쓰고, 읽은 책 내용을 요약하고, [[ ]]로 관련 노트끼리 직접 링크 걸고, 태그를 달고 — 이 모든 걸 **사람이 직접 타이핑**해서 만듭니다. 이게 "일반적인 Obsidian 사용법"입니다.
- 이 경우엔 LLM이 개입하지 않으니 "누가 부기를 대신 해주느냐"의 이점이 없고, 그냥 사람이 스스로 정리하는 디지털 메모장/제텔카스텐(Zettelkasten) 노트 앱일 뿐입니다.

### 그래서 두 가지를 비교하면

| 구분                | 일반 Obsidian 볼트  | LLM Wiki (카파시 패턴)   |
|-------------------|-----------------|---------------------|
| 페이지는 누가 쓰나        | 사람이 직접 타이핑      | LLM이 원본 소스를 읽고 작성   |
| 링크는 누가 거나         | 사람이 직접 [[ ]] 입력 | LLM이 자동으로 판단해서 삽입   |
| 유지보수(모순 체크, 업데이트) | 사람이 직접 해야 함     | LLM이 lint로 대신 처리    |
| Obsidian의 역할      | 편집기 + 뷰어        | 뷰어(읽기 전용으로 쓰는 게 권장) |

즉 **"일반 Obsidian 볼트"란 그냥 여느 사람들이 흔히 쓰는 평범한 개인 노트 앱으로서의 Obsidian 사용 방식**을 말한 거고, 만약 LLM Wiki에서 사람이 페이지를 직접 손으로 고치기 시작하면 결국 이 "평범한 사용법"으로 되돌아가는 셈이라 LLM에게 부기를 맡기는 이점이 사라진다는 뜻으로 말씀드린 겁니다.

## Q15. 사용자

---

부기 라는 용어가 익숙한게 아닌데 뭐야?

### A15. 어시스턴트(Claude)

---

**부기(簿記, bookkeeping)**는 원래 회계 용어로, 장부(帳簿)에 거래 내역을 하나하나 기록하고 정리하는 일을 뜻합니다. 회사에서 매일 들어오고 나가는 돈을 장부에 적고, 잔액을 맞추고, 항목별로 분류하는 반복적이고 꼼꼼한 사무 작업을 가리키는 말입니다.

여기서는 이런 뜻으로 썼습니다

LLM Wiki 맥락에서 "부기"는 실제 회계가 아니라, **"지식을 관리하는 데 필요한 반복적이고 꼼꼼한 정리 작업"**을 비유적으로 표현한 말입니다. 구체적으로는:

- 새 자료가 들어왔을 때 관련 페이지들을 찾아서 업데이트하기
- 페이지 간 링크를 걸고 유지하기
- 요약물 최신 상태로 유지하기
- 새 자료가 기존 내용과 모순되면 표시해두기
- 여러 페이지에 걸쳐 일관성을 유지하기

이런 작업들이 마치 회계 장부를 매일 꼼꼼히 기록하고 맞춰나가는 것과 비슷하다고 해서 "부기"라는 표현을 쓴 겁니다. 지식베이스를 유지하는 데 있어 지루한 부분은 읽거나 사고가 아니라 부기(bookkeeping)이고, 사람들이 위키를 포기하는 이유는 이 유지 관리 부담이 가치보다 더 빨리 커지기 때문이라는 게 카파시의 원래 표현입니다.

쉽게 바꿔 말하면

"부기"라는 말 대신 그냥 **"뒤치다꺼리"** 또는 **"자잘한 정리·유지보수 작업"**이라고 생각하시면 됩니다. 창의적이고 중요한 판단(무엇을 넣을지, 어떻게 해석할지)은 사람이 하고, 그 판단 이후에 따라오는 지루하고 반복적인 정리 작업(링크 걸기, 업데이트하기, 일관성 맞추기)은 LLM이 대신 해준다는 뜻입니다.

## Q16. 사용자

---

LLM Wiki의 구축과 활용의 전체적인 프로세스를 정리해 줘. 이때 기본적인 개념을 꼭

확인해 줘

## A16. 어시스턴트(Claude)

### LLM Wiki 구축 및 활용 전체 프로세스 정리

지금까지 나눈 내용을 처음부터 끝까지 순서대로 정리해 드리겠습니다. 먼저 기본 개념부터 다시 확인하겠습니다.

#### 0. 기본 개념 확인

| 용어                     | 의미                                                                  |
|------------------------|---------------------------------------------------------------------|
| <b>Gist</b>            | GitHub에서 짧은 글/코드를 간단히 공유하는 작은 게시글. 카파시가 이 형태로 아이디어를 올림              |
| <b>큐레이션(curation)</b>  | 아무 자료나 넣는 게 아니라, 사람이 직접 골라서 선별한 자료                                  |
| <b>마크다운(Markdown)</b>  | #, **처럼 간단한 기호로 문서 형식을 만드는 텍스트 문법. 확장자 .md                          |
| <b>위키(Wiki)</b>        | 위키백과처럼 여러 페이지가 링크로 서로 연결된 문서 모음                                     |
| <b>컴파일(compile)</b>    | 원본 자료를 읽고 정리해서 체계적인 위키 페이지로 만들어내는 과정                                |
| <b>엔티티(entity) 페이지</b> | 사람·회사·개념처럼 구체적인 대상 하나마다 만든 전용 페이지                                   |
| <b>종합(synthesis)</b>   | 여러 자료를 읽고 내린 나만의 결론/분석. 새 자료가 이를 뒷받침하면 "강화", 반박하면 "도전"              |
| <b>부기(bookkeeping)</b> | 링크 걸기, 요약 갱신, 모순 표시 같은 반복적·꼼꼼한 유지관리 작업 (회계 장부 정리에서 온 비유)            |
| <b>RAG와의 차이</b>        | RAG는 질문할 때마다 원본에서 재검색 → 추적 없음. LLM Wiki는 한번 정리한 지식이 위키에 누적되어 계속 유지됨 |

#### 1. 3 단계 아키텍처 (무엇이 어디에 있는가)

```
raw/  (원본 소스, 불변)  →  wiki/  (LLM이 쓰는 산출물)
      ↑
    CLAUDE.md (규칙/스키마)
```

- **Raw sources:** 사람이 큐레이션해서 넣은 원본 문서. LLM은 읽기만 하고 절대 수정 안 함 (진실의 원천)

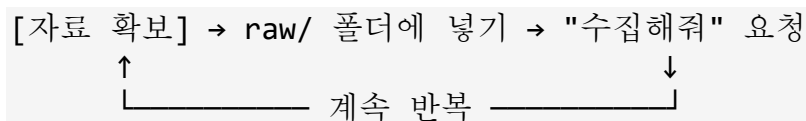
- **Wiki**: LLM 이 전적으로 소유하고 쓰는 마크다운 페이지들 (요약, 엔티티 페이지, 개념 페이지, 종합 페이지)
- **Schema (CLAUDE.md)**: 위키 구조와 규칙, 워크플로우를 LLM 에게 알려주는 설정 문서

## 2. 전체 프로세스 흐름

### 1 단계 — 최초 설정 (딱 1 회)

18. 카파시의 LLM Wiki 아이디어 문서를 Claude Code 에게 전달
19. 원하는 주제/목적을 설명 (예: "GD RDM 프로젝트 CSV 스키마 위키")
20. Claude Code 가 질문 → 답변 → **폴더 구조**(`raw/`, `wiki/`, `CLAUDE.md`) **자동 생성**
21. 필요하면 구조나 규칙을 사람이 검토·조정

### 2~3 단계 — Ingest 루프 (자료가 생길 때마다 반복)



- 사람: 어떤 자료를 넣을지 큐레이션 + ingest 명령
- LLM: 자료를 읽고 → 관련 엔티티/개념 페이지 생성 또는 업데이트 → 링크 자동 연결 → 인덱스·로그 갱신

### Query (질의) — 아무 때나 가능

- 사람이 위키에 질문 → LLM 이 관련 페이지를 읽고 답 종합
- 좋은 답변은 "이것도 위키에 저장해줘"라고 요청하면 새 페이지로 축적됨

### Lint (점검) — 주기적으로

- "위키 점검해줘" 요청 → LLM 이 모순, 고아 페이지, 누락된 연결 등을 보고
- 실제로 무엇을 고칠지는 사람이 최종 판단

### 3. 지원 소스 포맷

| 구분     | 형식                           |
|--------|------------------------------|
| 텍스트    | md, txt, csv, json, 코드 파일    |
| 문서     | pdf, docx, pptx, xlsx, html  |
| 이미지    | png, jpg, gif, webp, 스캔본 PDF |
| 오디오/영상 | 직접 지원 안 됨 → 텍스트로 먼저 변환 필요    |
| 압축     | zip 등 → 풀어서 개별 파일로 넣는 게 나음   |

기준: "글자로 읽히거나 눈으로 보이는 것"은 대부분 가능

### 4. Obsidian 의 역할

| 역할           | 설명                                             |
|--------------|------------------------------------------------|
| 뷰어/탐색기       | 그래프 뷰, 백링크, 검색으로 위키 구조를 시각적으로 확인               |
| (기술적으로만) 편집기 | 직접 타이핑 수정도 가능하지만, 권장 방식은 아님                    |
| 문제 발견 도구     | 이상한 링크·고아 페이지를 눈으로 발견하면 Claude Code에게 말로 수정 요청 |

원칙: **"사람은 읽기만, LLM이 편집"** — 링크 연결도 LLM이 자동 판단해서 겁니다. 코어 플러그인(Backlinks, Graph view, Outgoing links, Search, Properties)만 켜면 최소 요건 충족, Dataview·Marp는 처음부터 추가해도 무방합니다.

### 5. 위키 분리 기준

주제/목적별로 별도 폴더(=별도 Obsidian 볼트)로 만드는 게 기본입니다. 판단 기준은 **"두 주제의 페이지들이 실제로 서로 링크될 일이 있는가"** — 없으면 분리, 시너지가 있으면 통합.

## 6. 전체 개입 지점 총정리 (사람 vs LLM)

| 단계          | 사람이 하는 일              | LLM이 하는 일            |
|-------------|-----------------------|----------------------|
| 최초 설정       | 아이디어 전달, 주제 설명, 구조 검토 | 폴더 구조·스키마 작성         |
| 자료 확보       | 큐레이션(무엇을 넣을지 선택)      | (없음, 사람이 넣어야 시작됨)    |
| Ingest      | "수집해줘" 명령, 결과 검토      | 읽기, 페이지 생성/갱신, 링크 연결 |
| Query       | 질문 던지기, 좋은 답 저장 지시    | 관련 페이지 찾아 종합 답변      |
| Lint        | 점검 명령, 최종 수정 여부 결정    | 모순·고아 페이지 탐지 및 보고    |
| Obsidian 사용 | 그래프·백링크로 관찰, 이상하면 재지시 | 백그라운드에서 실시간 반영       |

**한 줄 요약:** 사람이 "무엇을 넣고, 언제 처리하고, 최종적으로 맞는지"를 판단하고, LLM이 "읽기·정리·링크·일관성 유지"라는 반복 노동을 전담하는 반자동 지식관리 시스템입니다.

### Q17. 사용자

이 세션의 내용을 docx로 저장해 줘. 내용의 축약이나 변경없이 보기 좋게만 저장해 줘

#### A17. 어시스턴트(Claude)

이번 세션의 전체 대화 내용을 축약이나 생략 없이 그대로 담은 Word 문서를 만들어 드렸습니다. 표, 코드 블록 등 원문 서식도 그대로 유지했습니다.

### Q18. 사용자

위키는 영구적이고 복리로 쌓이는 산출물이다. 상호참조는 이미 존재하고, 모순은 이미 표시되어 있으며, 라고 했는데 모순은 어떻게 처리해야 하는거야? LLM은 이걸 알려만 주는거고 처리는 전적으로 사용자의 몫이야?

#### A18. 어시스턴트(Claude)

좋은 질문입니다. 정리하면 "LLM이 알려만 주는 것"과 "LLM이 처리하는 것"이 단계별로 나뉘어 있습니다. 하나씩 짚어드리겠습니다.

1 단계 — LLM 이 자동으로 하는 것: 모순을 "위키에 기록"

여기서 중요한 포인트는, LLM이 모순을 단순히 채팅창에서 사람에게 "이거 모순인데요?"라고 말로만 알려주는 게 아니라는 겁니다. **위키 페이지 자체에 그 모순을 텍스트로 남겨둡니다.** 예를 들면:

#### ## 배터리 수명

2024년 논문 A는 리튬 배터리 수명이 평균 8년이라고 주장한다.

반면 2026년 논문 B는 신소재 적용 시 12년까지 가능하다고 보고했다.

[모순 노트] 두 수치 차이는 측정 조건(충방전 사이클 기준)이 다르기 때문일 수 있음 - 추가 검증 필요

이렇게 **모순 자체가 위키의 콘텐츠가 됩니다.** 이게 카파시가 "모순은 이미 표시되어 있다"고 한 말의 실제 의미입니다 — 다음에 이 페이지를 읽거나 질의할 때, 사람이나 LLM 둘 다 이 긴장 관계를 이미 알고 시작하는 것이지, 매번 새로 발견해야 하는 게 아닙니다.

이 기록이 일어나는 시점은 두 가지입니다:

- **Ingest 시점:** 새 자료를 넣었는데 기존 위키 내용과 어긋나면, 그 자리에서 바로 기록
- **Lint 시점:** 주기적 점검에서 이미 있던 페이지들 사이의 모순을 뒤늦게 발견해서 기록

## 2 단계 — 최종 판단은 사람의 몫

말씀하신 대로, **"어느 쪽이 맞다고 최종적으로 결론 내리는 것"은 전적으로 사람의 판단입니다.** LLM은 "이렇게 다른 두 주장이 있다"는 사실을 정리해줄 뿐, 어느 쪽이 옳은지 임의로 정하지 않습니다. 이걸 앞서 말씀드린 원칙 — LLM은 편집자가 아니라 제안자다 — 과 정확히 같은 맥락입니다.

## 3 단계 — 하지만 사람이 "혼자" 처리하는 건 아닙니다

여기서 오해하기 쉬운 부분인데, "처리는 전적으로 사용자의 몫"이라고 해서 사람이 아무 도움 없이 판단해야 하는 건 아닙니다. 실제로는 사람이 LLM에게 **판단 재료를 더 모아달라고 시킬 수 있습니다:**

- "이 두 논문 중 어느 게 더 최신 연구인지, 방법론 차이가 뭔지 다시 비교해줘" → LLM 이 웹 검색이나 두 소스를 재분석해서 판단 재료를 더 줌

- "논문 B가 더 신뢰할 만한 저널이니까, 위키에서는 B 기준으로 종합을 다시 써줘" → 사람이 판단을 내리면, 그 결정을 LLM에게 명령해서 실제 반영(다시 쓰기)은 LLM이 함
- 만약 사람이 아직 결론을 못 내렸다면 → 굳이 지금 해결 안 하고 "미해결 모순"으로 놔둔 채 위키에 표시만 해두고 넘어가도 됩니다 (나중에 또 다른 소스가 들어오면 자동으로 다시 걸림)

## 정리

| 단계                | 누가 하나                              |
|-------------------|------------------------------------|
| 모순 발견·기록          | LLM (ingest/lint 시점 자동)            |
| "어느 쪽이 맞는지" 최종 결론 | 사람                                 |
| 결론을 근거로 위키 재작성    | LLM (사람이 명령하면)                     |
| 결론이 안 나면?         | 그냥 "미해결 모순"으로 표시된 채 위키에 남겨둠 — 문제없음 |

즉 "발견하고 기록하는 건 LLM이 자동으로 하고, 판단은 사람이 하되, 판단에 필요한 조사와 실제 반영 작업은 다시 LLM에게 시킬 수 있다"는 구조입니다. 사람이 조사부터 수정까지 혼자 다 해야 하는 건 아니고, "이거 어느 쪽이 맞는지 알아봐줘" 하고 LLM에게 다시 던질 수 있습니다.

## Q19. 사용자

카파시 본인은 위키를 직접 쓰는 경우가 거의 없고, LLM이 모든 것을 작성하고 유지합니다. 본인은 소싱, 탐색, 좋은 질문을 던지는 것을 담당합니다. 라고 했는데 질문의 결과(답변)을 LLM이 할건데 이것도 Wiki에 자동으로 반영되어 산출물로 업데이트 되는 거야?

## A19. 어시스턴트(Claude)

아니요, **자동으로 반영되지는 않습니다.** 이 부분은 명확히 짚고 넘어가는 게 좋겠습니다.

기본 원칙 — Query 는 "저장이 아니라 그냥 답변"

Query(질의)는 원래 성격 자체가 "지금 위키에 쌓인 걸 가지고 질문에 답하는 것"이지, "위키를 수정하는 작업"이 아닙니다. LLM이 관련 페이지들을 읽고 종합해서 답을 채



팅창에 보여주는 것으로 끝나며, 이 답변 자체는 기본적으로 **대화 기록에만 남고 위키 파일에는 반영되지 않습니다.**

그럼 왜 "좋은 답변은 위키에 파일링될 수 있다"고 했나?

여기서 "될 수 있다(can be)"는 자동으로 일어난다는 뜻이 아니라, **사람이 "이것도 저장해줘"라고 명시적으로 요청해야 일어나는 선택적 행동**입니다. 이전에 정리해드린 사람 개입 지점 표에서도 이 부분을 짚었었습니다 — Query 단계에서 사람이 하는 일 중 하나가 "좋은 답변 파일링 결정"입니다.

실제 흐름 예시

**저장 안 하는 경우 (기본값):**

사람: "지금까지 모은 자료 기준으로 A와 B 배터리 방식을 비교해줘"

LLM: [비교 답변을 채팅으로 제공]

→ 여기서 끝. 위키 파일은 하나도 안 바뀔. 다음에 똑같이 물어보면 LLM이 또 처음부터 다시 종합해야 함.

**저장하는 경우 (사람이 명시적으로 지시):**

사람: "지금까지 모은 자료 기준으로 A와 B 배터리 방식을 비교해줘"

LLM: [비교 답변을 채팅으로 제공]

사람: "이 비교 내용 위키에 페이지로 저장해줘"

LLM: → `wiki/A-B-배터리-비교.md` 라는 새 페이지를 만들어서 저장

→ 다음부터는 이 페이지가 위키에 남아있어서, 관련 질문이 오면 LLM이 매번 재종합하지 않고 이 페이지를 참조할 수 있음

왜 자동 저장으로 안 만들었을까

이건 의도적인 설계로 보입니다:

- **모든 질문의 답이 다 가치 있는 건 아닙니다.** 가벼운 확인성 질문("이 논문 몇 년도 거야?")까지 전부 위키 페이지로 만들면 위키가 잡음으로 가득 찹니다.
- **"이건 저장할 가치가 있다"는 판단 자체가 사람의 큐레이션 역할**입니다. 앞서 말씀드린 "사람이 하는 일" 목록의 연장선입니다 — 무엇을 넣을지 고르는 게 raw sources 뿐 아니라 질의 결과에도 똑같이 적용되는 것입니다.

## 정리

|                              | 자동 저장?                  |
|------------------------------|-------------------------|
| 원본 소스 → 위키 페이지 (ingest)      | 자동 아님, "수집해줘" 명령 필요     |
| 질의 답변 → 위키 페이지 (query 결과 저장) | 자동 아님, "이것도 저장해줘" 명령 필요 |

즉 **ingest도 query도 똑같은 원칙**을 따릅니다 — LLM은 요청받은 것만 위키에 반영하고, "이걸 반영할지 말지"는 항상 사람이 결정합니다. 다만 실무적으로는 스키마 (CLAUDE.md)에 "이런 종류의 질문 답변은 자동으로 위키에 저장해줘" 같은 규칙을 미리 정해두면, 매번 "저장해줘"라고 말하지 않아도 원하는 패턴대로 자동화할 수도 있습니다 — 이걸 어디까지나 사람이 스키마 설정 단계에서 미리 정해두는 규칙이라, 여전히 "사람의 결정"이 근본 출처인 셈입니다.

## Q20. 사용자

"Obsidian Web Clipper 브라우저 확장을 쓰면 웹 아티클을 마크다운으로 변환해 raw 폴더에 바로 저장할 수 있어 소스 수집이 빨라집니다.

[GitHub](https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f)

\* 위키가 커지면(수백 페이지 이상) 인덱스 파일만으로는 부족해질 수 있는데, 이때는 qmd 같은 로컬 하이브리드(BM25/벡터) 검색 도구를 CLI나 MCP 서버 형태로 붙이는 것을 고려" 에서 Obsidian Web Clipper 브라우저 확장이 어떤 브라우저용이 있어? qmd 는 뭐고, 로컬 하이브리드(BM25/벡터) 검색 도구를 CLI나 MCP 서버 형태로 붙이는 것이 뭔지 잘 모르겠어. 자세히 알려줘

## A20. 어시스턴트(Claude)

하나씩 자세히 풀어드리겠습니다

### 1. Obsidian Web Clipper — 지원 브라우저

공식 확장 프로그램은 주요 브라우저를 대부분 지원합니다.

| 브라우저 계열            | 지원 브라우저                           | 설치처                                       |
|--------------------|-----------------------------------|-------------------------------------------|
| <b>Chromium 계열</b> | Chrome, Brave, Edge, Arc, Orion 등 | Chrome 웹 스토어 (Edge는 Edge Add-ons에서 별도 제공) |

| 브라우저 계열        | 지원 브라우저                               | 설치처                       |
|----------------|---------------------------------------|---------------------------|
| <b>Firefox</b> | Firefox (데스크톱), Firefox Mobile(안드로이드) | Firefox Add-ons           |
| <b>Safari</b>  | macOS, iOS, iPadOS                    | App Store 내 Safari 확장 갤러리 |

즉 크롬 계열, 파이어폭스, 사파리 세 축을 다 지원하고, 모바일(안드로이드 파이어폭스, iOS 사파리)에서도 쓸 수 있습니다. 참고로 Obsidian 앱 자체가 1.7.2 이상 버전이어야 합니다.

## 2. qmd 가 뭐야?

**qmd**는 Tobi Lütke(Shopify 창업자)가 만든 오픈소스 도구로, 정식 이름은 "Query Markdown"입니다. 한마디로 **"내 컴퓨터 안에 있는 마크다운 파일들을 빠르고 똑똑하게 검색해주는 프로그램"**입니다.

핵심 특징:

- **완전히 로컬에서 동작합니다** — 인터넷에 데이터를 전송하지 않고 내 컴퓨터 안에서만 검색이 이루어집니다. 이게 "온디바이스(on-device)"라는 표현의 의미입니다.
- 위키가 커져서(수백 페이지 이상) 단순히 Ctrl+F 로 찾기엔 너무 방대해질 때, "이 내용과 관련된 페이지를 찾아줘"라고 하면 정확도 높게 찾아주는 검색엔진입니다.
- 결과가 하나의 SQLite 파일(가벼운 로컬 데이터베이스)로 저장되어서 별도 서버 설치 없이 그냥 바이너리 하나로 작동합니다.

## 3. "하이브리드(BM25/벡터) 검색"이 뭐야?

이건 qmd가 검색할 때 세 가지 방식을 동시에 섞어서 쓴다는 뜻입니다. 각각 설명드리면:

### BM25 — 키워드 매칭 검색

**정확한 단어가 문서에 있는지**를 찾는 전통적인 검색 방식입니다. 예를 들어 "리튬 배터리"라고 검색하면 그 단어가 실제로 들어간 문서를 찾아줍니다. 구글 이전 시대의 검색

엔진들이 주로 쓰던 방식이고, 정확한 용어·고유명사·프로젝트 코드명 검색에 강합니다.

**한계:** "리튬 배터리"라고 검색했는데 어떤 페이지에 "전고체 셀"이라고만 적혀 있으면, 실제로는 관련 있는 내용인데도 단어가 다르면 못 찾습니다.

### 벡터(vector) 검색 — 의미 기반 검색

단어 자체가 아니라 **"의미가 비슷한지"**를 계산해서 찾는 방식입니다. AI가 문장을 숫자로 변환(임베딩)해서, "리튬 배터리 열화"와 "전고체 셀 수명 저하"처럼 단어는 다르지만 의미가 비슷한 문서까지 찾아줍니다.

**한계:** 반대로 너무 의미 위주로만 찾다 보면, 정확히 그 단어를 찾고 싶을 때(예: 특정 프로젝트 코드명) 오히려 놓칠 수 있습니다.

### 하이브리드 = 이 둘을 합친 것

qmd는 BM25(정확한 키워드)와 벡터 검색(의미 유사도)을 **둘 다 돌린 다음, 결과를 합쳐서(RRF라는 방식으로 순위를 재조합) 최종 결과를 냅니다.** 여기에 한 단계 더해서, **LLM이 상위 결과들을 다시 한번 검토해서 순위를 조정(재순위화)**합니다. 즉 "단어도 맞고 의미도 맞는" 결과를 가장 위로 올려주는 3중 안전장치라고 보시면 됩니다.

## 4. "CLI 나 MCP 서버 형태로 붙인다"는 게 뭐야?

이건 **qmd를 Claude Code 같은 LLM 에이전트에게 연결하는 두 가지 방법**을 말한 겁니다.

### CLI(Command Line Interface) 방식

qmd를 그냥 터미널 명령어처럼 설치해두는 방식입니다. 예를 들면:

```
qmd search "리튬 배터리 열화"
```

이렇게 터미널에서 직접 칠 수 있는 명령어입니다. Claude Code는 bash 도구로 터미널 명령을 실행할 수 있으니, "qmd로 이 개념 검색해줘"라고 하면 **Claude Code가 대신 이 명령어를 터미널에서 실행(셀아웃)해서** 결과를 가져옵니다.

## MCP(Model Context Protocol) 서버 방식

MCP는 Anthropic이 만든 프로토콜로, LLM이 외부 도구를 "자연스러운 기능"처럼 직접 호출할 수 있게 해주는 연결 방식입니다. qmd를 MCP 서버로 띄워두면, Claude Code가 터미널 명령어를 흉내 내는 게 아니라 **원래부터 갖고 있던 도구처럼 qmd 검색 기능을 바로 호출**할 수 있습니다.

### 둘의 차이

|        | CLI 방식                      | MCP 서버 방식                    |
|--------|-----------------------------|------------------------------|
| 연결 방법  | Claude Code가 터미널 명령어를 대신 실행 | Claude Code가 네이티브 도구처럼 직접 호출 |
| 설정 난이도 | 상대적으로 간단                    | 초기 설정이 조금 더 필요               |
| 통합도    | "명령어를 대신 쳐주는" 느낌            | "원래 기능인 것처럼" 매끄러운 느낌         |

실제로는 qmd 하나를 설치하면 두 가지 방식 모두 제공하기 때문에, 사용자가 상황에 맞게 아무거나 골라 쓰면 됩니다. 위키가 아주 커지기 전까지는(수십~백 페이지 수준) 이런 도구 없이 `index.md`만으로 충분하니, 당장 필요하신 단계는 아니고 "위키가 정말 커졌을 때 고려할 수 있는 옵션" 정도로 기억해두시면 됩니다.

## Q21. 사용자

"7단계 — 첫 소스 수집(ingest)

`raw/` 폴더에 첫 문서(PDF, txt, md 등)를 넣고 Claude Code에게 "이 소스를 수집해줘"라고 요청합니다." 라고 했는데 이미 `raw/` 폴더에 여러개의 파일들이 있다면 내가 방금 집어 넣은 파일들을 이전 파일들과 구분해서 알아서 방금 넣은 파일들만 수집하는거야?

## A21. 어시스턴트(Claude)

좋은 질문입니다. 결론부터 말씀드리면, "저절로 알아서 구분한다"기보다는, 위키의 `log.md`(로그) 파일을 기준으로 "이미 처리한 것"과 "아직 처리 안 한 것"을 비교해서 구분합니다. 완전 자동 감지가 아니라, 기록을 근거로 판단하는 방식입니다.

왜 로그 파일이 필요한가

raw/ 폴더에 파일이 10개 있다고 해서, LLM이 파일 생성 시간이나 "새로 추가됐다"는 표시를 마법처럼 아는 게 아닙니다. LLM이 매번 새 대화 세션을 시작할 때는 "이 폴더에 뭐가 있었는지, 뭘 이미 처리했는지"에 대한 기억이 없습니다(대화가 끝나면 리셋되기 때문입니다). 그래서 `log.md`가 "무엇을 언제 수집했는지"를 기록해두는 역할을 합니다.

예를 들어 log.md에 이렇게 쌓여 있다면:

```
## [2026-07-15] ingest | 논문A.pdf
## [2026-07-20] ingest | 논문B.pdf
## [2026-07-28] ingest | 회의록1.md
```

이 상태에서 사용자가 "이 소스를 수집해줘"라고 말하면, LLM은 raw/ 폴더의 전체 파일 목록과 log.md에 이미 기록된 파일 목록을 대조해서 "아, 논문A/B/회의록1은 이미 처리됐고, 나머지가 새로 넣은 것이구나"라고 추론합니다.

그런데 이 방식이 100% 정확하진 않습니다 — 그래서 명시적으로 말씀하시는 게 안전합니다

이 log.md 대조 방식은 스키마(CLAUDE.md)에 "매번 raw 폴더와 log.md를 비교해서 처리 안 된 파일만 골라라"는 규칙을 명시적으로 넣어뒀을 때만 안정적으로 작동합니다. 이 규칙이 없거나 애매하면, LLM이 실수로 이미 처리한 파일을 다시 처리하거나(중복 작업), 반대로 새 파일을 놓칠 수도 있습니다.

그래서 실무적으로는 두 가지 방법을 권장드립니다.

### 방법 1 — 파일명을 직접 언급하기 (가장 안전)

"방금 raw/ 폴더에 넣은 '리튬배터리\_2026.pdf' 수집해줘"

이렇게 파일명을 꼭 집어 말하면, LLM이 헛갈릴 여지가 전혀 없습니다.

### 방법 2 — 스키마에 규칙을 정해두고 "새 파일들 수집해줘"라고 포괄적으로 요청하기

CLAUDE.md에 "ingest 요청을 받으면 항상 raw/ 폴더와 log.md를 비교해서 아직 로그에 없는 파일들만 처리하라"는 규칙을 최초 설정 시 넣어두면, "새로 넣은 파일들 수집해

줘"라고만 말해도 알아서 비교·판단해서 처리합니다. 다만 이 경우도 처리가 끝난 뒤 log.md나 위키 인덱스를 한 번 확인해서 의도한 파일만 처리됐는지 검토하시는 게 안전합니다.

## 정리

|           | 설명                                                    |
|-----------|-------------------------------------------------------|
| 완전 자동 감지? | 아니요 — "새 파일이다"라는 정보를 스스로 아는 게 아니라 log.md와 대조해서 추론하는 것 |
| 정확도를 높이려면 | 스키마에 "raw/ vs log.md 비교" 규칙을 미리 정의해두기                 |
| 가장 확실한 방법 | 파일명을 직접 언급해서 요청하기                                     |

처음 안내드린 "7단계"는 raw/ 폴더가 비어있는 상태에서 첫 파일 하나만 넣는 상황이라 이런 구분 문제가 아예 생기지 않았던 것이고, 지금 질문하신 것처럼 **파일이 여러 개 쌓인 이후부터는 이 구분 로직이 실제로 중요해지는 지점**입니다. 좋은 포인트를 짚어 주셨습니다.

## Q22. 사용자

"단계 — Obsidian 설치

공식 사이트(obsidian.md)에서 Windows 인스톨러를 받아 설치한 뒤, "Open folder as vault"로 C:\claude\projects\my-wiki를 엽니다. 이후 이미지 자동 다운로드를 원하시면 Settings → Files and links에서 "Attachment folder path"를 raw/assets/로 고정하고, Settings → Hotkeys에서 "Download attachments for current file"에 단축키(예: Ctrl+Shift+D)를 지정해두시면 편리합니다." 라는 설명에서 1. 이미지가 뭐야? 2. 이미지 자동 다운로드가 뭐야? 3. raw/ 밑에 assets/ 폴더는 내가 직접 만들어야 해? 4. 궁극적으로 "이후 이미지 ~~" 부분이 설명하는게 뭐야?

## A22. 어시스턴트(Claude)

하나씩 풀어드리겠습니다

### 1. "이미지"가 뭐야?

여기서 말하는 이미지는 웹 아티클을 클리핑(저장)할 때 그 페이지 안에 들어있는 사

진, 스크린샷, 다이어그램, 그래프 같은 시각 자료를 말합니다. 예를 들어 어떤 기술 블로그 글을 Web Clipper로 저장하면, 그 글 안에 있던 아키텍처 다이어그램이나 제품 사진도 함께 딸려옵니다. 그게 여기서 말하는 "이미지"입니다.

## 2. "이미지 자동 다운로드"가 뭐야?

이걸 이해하려면 먼저 **문제 상황**을 아셔야 합니다.

**문제:** 웹 아티클을 클리핑하면, 기본적으로 이미지는 파일 자체가 복사되는 게 아니라 **원래 웹사이트의 이미지 주소(URL)를 그대로 링크만 걸어놓는 방식**으로 저장됩니다. 예를 들면 이런 식입니다:

```
![다이어그램](https://example.com/images/diagram123.png)
```

문제는 이 방식이면, 나중에 원래 웹사이트에서 그 이미지를 지우거나 사이트 자체가 없어지면, 내 위키에서도 이미지가 깨져서 안 보이게 됩니다. 즉 내 컴퓨터에 저장된 게 아니라 남의 서버에 의존하고 있는 상태인 겁니다.

**해결책 = 이미지 자동 다운로드:** 이 원격 URL 이미지를 내 컴퓨터 디스크로 실제로 다운로드해서, 링크를 로컬 파일 경로로 바꿔치기하는 기능입니다. 이렇게 하면:

- 원본 사이트가 없어져도 내 위키에서는 이미지가 계속 보임
- 인터넷 연결 없이 오프라인에서도 이미지를 볼 수 있음
- LLM 이 이미지를 직접 눈으로 읽어서 분석할 때도, 원격 URL 보다 로컬 파일이 훨씬 안정적으로 처리됨

## 3. raw/assets/ 폴더는 직접 만들어야 해?

아니요, 직접 만들 필요 없습니다. Obsidian 설정에서 경로를 raw/assets/로 지정만 해두면, 실제로 처음 이미지를 다운로드하는 순간 그 폴더가 존재하지 않으면 **Obsidian이 자동으로 만들어줍니다**. 즉 미리 빈 폴더를 만들어두는 수고는 안 하셔도 되고, 설정에 경로만 적어두시면 됩니다.

## 4. 결국 이 문단 전체가 설명하는 것



정리하면 이 부분은 "웹에서 클리핑한 아티클의 이미지가 깨지지 않고 항상 내 컴퓨터에 실제로 저장되도록 만드는 설정 두 가지"를 안내한 것입니다.

| 설정                                              | 하는 일                                                                  |
|-------------------------------------------------|-----------------------------------------------------------------------|
| ① Attachment folder path를 raw/assets/로 지정       | 앞으로 다운로드되는 이미지들이 흩어지지 않고 이 폴더 한 곳에 모이도록 지정                            |
| ② Download attachments for current file에 단축키 지정 | 아티클을 클리핑한 뒤, 단축키 한 번(Ctrl+Shift+D)으로 그 노트 안의 모든 원격 이미지를 한 번에 로컬로 다운로드 |

즉 흐름은 이렇습니다:

1. Web Clipper로 웹 아티클 클리핑  
→ raw/ 폴더에 .md 파일 생성됨 (이미지는 아직 원격 URL 링크 상태)
2. 단축키(Ctrl+Shift+D) 누름  
→ "Download attachments for current file" 명령 실행  
→ 원격 이미지들이 raw/assets/ 폴더로 실제 다운로드되고  
→ 문서 안의 링크가 로컬 경로로 자동 교체됨

이 과정을 거쳐야 나중에 LLM이 raw/ 폴더를 ingest할 때 이미지도 안정적으로 함께 읽어서 분석할 수 있는 상태가 됩니다. 참고로 이 명령어는 Obsidian 1.8.3 버전부터 추가된 기능이라, 너무 오래된 버전을 쓰고 계시면 업데이트가 필요할 수 있습니다.

## Q23. 사용자

LLM이 Raw Resource에서 자료를 읽어서 Wiki를 만들면 그 결과물들은 모두 .md 파일로만 생성되는거야?

## A23. 어시스턴트(Claude)

네, 위키 계층(wiki/ 폴더) 안에 LLM이 새로 만들어내는 파일은 기본적으로 전부 .md(마크다운) 파일입니다. 다만 몇 가지 예외적인 경우와 함께 정리해 드리겠습니다.

### 기본 원칙 — 왜 .md 만 쓰는가

앞서 말씀드린 대로 위키는 "마크다운 파일들의 디렉터리"로 정의되어 있습니다. 엔티티 페이지든, 개념 페이지든, 종합 페이지든, index.md든, log.md든, 심지어 스키마 파일

(CLAUDE.md)까지도 전부 마크다운 형식입니다. 이렇게 한 가지 포맷으로 통일하는 이유는:

- Obsidian 이 마크다운을 기본으로 다루기 때문에 그래프 뷰, 백링크, 검색이 다 잘 작동함
- [[위키링크]] 문법 자체가 마크다운 생태계의 문법이라, 다른 포맷으로 쓰면 링크 연결이 깨짐
- 텍스트 파일이라 git 으로 버전 관리하기 쉽고, grep 같은 유닉스 도구로 빠르게 검색 가능

### 예외 — .md 가 아닌 결과물이 나올 수도 있는 경우

이건 위키 페이지 자체가 아니라, **Query(질의)에 답할 때 생성되는 부가 산출물**에 해당합니다.

| 상황                | 결과물 형식                                                                                               |
|-------------------|------------------------------------------------------------------------------------------------------|
| 일반적인 질문에 대한 답변 정리 | .md 페이지                                                                                              |
| "슬라이드로 만들어줘" 요청 시 | Marp 슬라이드 — 이것도 사실은 .md 파일인데, Marp 플러그인이 특수 문법을 읽어서 슬라이드 형태로 "렌더링"만 해주는 것. 원본 파일 자체는 여전히 마크다운입니다     |
| "차트로 그려줘" 요청 시    | matplotlib 등으로 그린 이미지 파일(.png 등) — 이 경우는 .md 가 아닌 실제 이미지 파일이 생성되고, 위키 페이지에서는 그 이미지를 ![]() 문법으로 참조만 함 |
| "캔버스로 정리해줘" 요청 시  | Obsidian의 Canvas 기능은 .canvas라는 별도 확장자를 씁니다 (JSON 기반 파일, 마크다운은 아님)                                    |

### 정리

**핵심 위키 콘텐츠(엔티티 페이지, 개념 페이지, 종합 페이지, 인덱스, 로그)는 100%**

**.md 파일**이고, 이게 원칙입니다. 다만 사람이 "이걸 슬라이드로", "이걸 차트로", "이걸 캔버스로" 같은 **특정 시각화 형태를 명시적으로 요청**하면, 그 결과물은 .md가 아닌 별도 형식(이미지, .canvas 등)으로 생성될 수 있고 위키 페이지 안에서는 그 파일을 참조하는 링크만 마크다운으로 남습니다.

즉 **"위키의 뼈대는 전부 마크다운, 그 위에 필요할 때만 붙는 부가 산출물(이미지·슬라이드·캔버스)이 예외적으로 다른 형식"**이라고 이해하시면 정확합니다.

## Q24. 사용자

---

소스가 계속 누적되면 LLM이 이것을 관리하다보면 결국 이에 비례해서 토큰 소모가 계속 누적해서 증가하는거야?

### A24. 어시스턴트(Claude)

---

좋은 질문입니다. 답은 **"완전히 비례해서 계속 증가하지는 않지만, 어느 정도는 확실히 늘어나고, 특히 두 지점에서 비용이 발생한다"**입니다. Ingest(수집)와 Query(질의)를 나눠서 설명드리는 게 정확합니다.

#### 1. Ingest(수집) 비용 — 위키 "크기"보다 "영향받는 페이지 수"에 비례

새 소스 하나를 ingest할 때 토큰이 드는 곳은:

- 그 소스 문서 자체를 읽는 비용
- 관련된 기존 페이지 10~15 개를 찾아서 읽는 비용 (예지 케이스에서는 실제로 소스 하나당 관련 페이지 10~15 개에 영향을 준다고 알려져 있습니다)
- 그 페이지들을 다시 쓰는(rewrite) 비용

여기서 중요한 포인트: 위키 전체를 매번 다 읽는 게 아니라, **"이 소스와 관련된 일부 페이지"만 읽고 고칩니다.** 그래서 위키가 100페이지든 500페이지든, 소스 하나를 ingest하는 비용 자체는 크게 안 늘어납니다 — 위키 **"총량"**이 아니라 **"이 소스가 몇 개 페이지에 영향을 주는가"**에 비례합니다.

다만 **"쌓아온 소스 개수"**에는 비례해서 늘어납니다. 소스를 100개 넣으면 ingest를 100번 한 것이므로, 누적 토큰 소모는 당연히 계속 늘어납니다. 이건 피할 수 없는 부분입니다. 실제로 논문 50편 정도를 수집해서 위키 페이지 10개를 만드는 경우, Claude API를 대략 100~200번 호출하게 된다는 보고도 있습니다.

#### 2. Query(질의) 비용 — 여기가 핵심 방어선

질문하실 때마다 위키 전체를 읽으면 당연히 위키가 커질수록 질의 비용도 계속 늘어나겠죠. 하지만 이것을 막기 위해 나온 게 바로 **index.md 패턴**입니다.

- LLM은 질문이 들어오면 **먼저 index.md(몇천 토큰 수준의 요약 카탈로그)만 읽고**, 거기서 관련 있어 보이는 페이지만 골라서 그 페이지들만 읽습니다.
- 즉 위키가 100 페이지든 1000 페이지든, **매 질의마다 실제로 읽는 양은 "질문과 관련된 몇 개 페이지 + 인덱스"로 거의 고정됩니다.**

이게 왜 중요하냐면, 실제 비교에서 이 방식이 전체 위키를 다 컨텍스트에 밀어넣는 방식보다 **토큰을 수십 배 절약**한 사례도 보고되고 있습니다 (한 사례에서는 71.5배 차이가 났다고 합니다). 카파시 본인의 위키가 100개 아티클, 40만 단어까지 커졌는데도 이 인덱스 방식이 여전히 잘 작동했다고 밝힌 것도 이 원리 덕분입니다.

### 3. 그래도 완전히 무한정 버티는 건 아닙니다 — 실질적인 한계선

몇 가지 현실적인 한계가 있습니다:

- **컨텍스트 윈도우가 아무리 커도(100 만 토큰급이라도), 실제로는 20 만~30 만 토큰을 넘어가면 품질이 떨어지기 시작**한다는 관찰이 있습니다. LLM이 연결을 놓치거나 일관성 없는 페이지를 만들어내기 시작합니다. 단순히 "답을 수 있냐"가 아니라 "답아도 잘 활용하느냐"의 문제입니다.
- 위키가 이 정도로 커지면, index.md 하나로도 관련 페이지를 정확히 못 찾는 지점이 옵니다. 이때가 바로 이전에 설명드린 **qmd 같은 로컬 검색 도구**를 붙여야 하는 시점입니다 — index.md를 통째로 읽는 대신, 검색 엔진이 미리 관련 페이지 후보를 좁혀주는 방식으로 전환하는 것입니다.

### 정리

| 구분            | 토큰 소모 추세                                           |
|---------------|----------------------------------------------------|
| Ingest 누적 비용  | 소스 개수에 비례해서 계속 증가 (피할 수 없음)                        |
| Ingest 1회당 비용 | 위키 총 크기와 무관, "이 소스가 건드리는 페이지 수"에만 비례               |
| Query 비용      | index.md 패턴 덕분에 위키가 커져도 거의 고정적으로 유지됨               |
| 한계 지점         | 위키가 20만~30만 토큰 규모를 넘으면 인덱스만으로 부족 → qmd 같은 검색 도구 필요 |

결론적으로, "**모든 걸 다 다시 읽는**" 방식이 아니기 때문에 RAG 없이 매번 재검색하는 것보다는 훨씬 효율적이지만, 소스를 계속 넣는 이상 누적 비용 자체는 늘어나며, 위키가 아주 커지면 검색 도구 없이는 결국 한계에 부딪힙니다. "누적되는 소스 개수"와 "위

키가 결국 도달하는 크기" 두 가지 모두가 장기적으로는 비용에 영향을 준다고 보시는 게 정확합니다.

## Q25. 사용자

---

그럼 이렇게 안드레이 카파시의 LLM Wiki를 운영하려면 기존 구독으로는 안되고 API 비용을 지불해야 하는거야? 결국 비용이 이중으로 지출되어야 하는거야?

## A25. 어시스턴트(Claude)

---

좋은 질문입니다. 결론부터 말씀드리면, **기존 Claude 구독(Pro/Max)만으로도 충분히 운영 가능하고, 별도로 API 비용을 추가로 내야 하는 게 아닙니다.** 다만 조건과 주의할 함정이 몇 가지 있습니다.

### 기본 구조 — Claude Code 는 구독 안에 포함됩니다

Claude Code는 별도로 판매되는 독립 상품이 아니라, **기존 Claude Pro/Max 구독에 포함된 기능**입니다. Pro 플랜(\$20/월)에 "Claude Code 포함"이라고 명시되어 있고, 터미널·VS Code·JetBrains·데스크톱 앱 어디서 실행하든 같은 구독 사용량 한도 안에서 작동합니다. 즉 최상길님이 이미 Pro나 Max를 쓰고 계시다면, **LLM Wiki를 위한 ingest/query/lint 작업도 그 구독 한도 안에서 그냥 돌아갑니다.** 별도 API 키를 발급받아 결제 정보를 따로 등록할 필요가 없습니다.

### 그럼 "이중 지출"이 생기는 경우는 언제인가

두 가지 경우에만 추가 비용이 발생합니다.

#### 1. 구독 한도를 다 써버렸을 때

Pro/Max는 5시간 단위로 갱신되는 세션 한도와 주간 한도가 있습니다. LLM Wiki의 ingest 작업은 앞서 말씀드린 대로 소스 하나당 여러 페이지를 다시 쓰는 작업이라 꽤 토큰을 먹는 편인데, 이 한도를 넘기면 두 가지 선택지가 주어집니다:

- 한도가 리셋될 때까지 기다리기 (추가 비용 없음)
- API 요금으로 초과분만 결제하기 (선택 사항, 강제 아님)

즉 **한도를 넘기지 않는 한 추가 비용은 전혀 없고**, 넘기더라도 "기다릴지, 초과분만 낼지"는 사용자가 고를 수 있습니다.

## 2. 환경변수 실수로 인한 의도치 않은 이중 과금 — 이걸 진짜 조심하셔야 합니다

만약 컴퓨터에 `ANTHROPIC_API_KEY`라는 환경변수가 설정되어 있으면, **Claude Code**는 구독을 무시하고 조용히 API 키로 과금을 전환합니다. 구독료(월 \$20~\$200)를 내고 있는데도, Claude Code는 그걸 안 쓰고 API로 별도 청구되는 상황이 생기는 겁니다. 실제로 이런 걸 모르고 쓰다가 "Max 구독료 \$200에 API 청구서 \$400이 따로 나왔다"는 사례가 보고되기도 했습니다.

**확인 방법** (Mac/Windows 터미널에서):

```
echo $ANTHROPIC_API_KEY
```

뭔가 출력되면, 그 값을 지우시거나(unset) "구독 그대로 쓰겠다"는 걸 확실히 하셔야 이중 과금을 피할 수 있습니다.

### 정리

| 상황                                            | 비용                               |
|-----------------------------------------------|----------------------------------|
| 구독 한도 안에서 ingest/query/lint 사용                | 추가 비용 없음, 구독료에 이미 포함             |
| 구독 한도를 초과했을 때                                 | 선택: 기다리거나(무료) 초과분만 API 요금 지불(선택) |
| <code>ANTHROPIC_API_KEY</code> 가 실수로 설정돼 있을 때 | 구독을 안 쓰고 전액 API로 과금됨 — 확인 필수     |

**결론적으로, LLM Wiki를 운영하기 위해 "당연히" API 비용을 이중으로 내야 하는 구조가 아닙니다.** 기존 구독만으로 시작하시고, 나중에 ingest 빈도가 많아져서 한도를 자주 넘기신다면 그때 Max 플랜으로 올리거나 초과분만 API로 결제할지 판단하시면 됩니다. 다만 시작하시기 전에 `ANTHROPIC_API_KEY` 환경변수가 설정돼 있지 않은지 한번 확인해보시는 걸 권해드립니다 — 이게 원인 모를 이중 과금의 가장 흔한 함정입니다.

## Q26. 사용자

너의 "실제로 논문 50편 정도를 수집해서 위키 페이지 10개를 만드는 경우, Claude API를 대략 100~200번 호출하게 된다는 보고도 있습니다." 설명을 보면 API를 호출하는 것처럼 설명이 되어 있어서 물어보는거야

## A26. 어시스턴트(Claude)

아, 좋은 지적이십니다. 제가 표현을 명확히 안 해서 혼란을 드렸네요. 정리해드리겠습니다.

### 그 "100~200 번" 수치는 어디서 나온 건가

그 수치는 카파시의 아이디어를 참고해서 누군가가 직접 만든 자동화 파이프라인(스크립트)을 분석한 사례에서 나온 것입니다. 이런 사람들은 Claude Code 터미널에서 대화하듯 쓰는 게 아니라, **직접 코드를 짜서 Anthropic API를 프로그램적으로 호출**하도록 자동화 도구를 만듭니다. 예를 들어 "소스 하나가 들어오면 자동으로 관련 페이지를 찾고(1번 호출) → 내용을 종합해서 페이지를 다시 쓰는(1번 호출) 스크립트"를 만든 겁니다. 이 경우는 **개발자용 API 키를 콘솔에서 발급받아 코드에 박아넣고 쓰는 방식**이라, 당연히 "API 호출"이라는 표현이 정확하고, 이걸 구독과 별개로 API 요금이 청구되는 경로입니다.

### 반면 최상길님이 실제로 하시게 될 방식은 다릅니다

지금까지 안내드린 워크플로우는 **Claude Code 터미널(또는 앱)에서 사람이 직접 대화하듯 "이 소스 수집해줘"**라고 타이핑하는 방식입니다. 이걸:

- 뒤에서 기술적으로는 모델을 "부르는" 게 맞습니다 (넓은 의미에서는 이것도 "API 호출"이라고 부를 수 있긴 합니다)
- 하지만 이 호출이 **구독(Pro/Max) 계정으로 로그인된 경로를 통해** 이루어지기 때문에, 앞서 설명드린 대로 **구독 사용량 한도 안에서** 처리됩니다
- 즉 같은 "호출"이라는 단어를 써도, **누구의 계정/키로** 청구되느냐가 완전히 다른 문제입니다

### 그러니까 헷갈리셨던 부분을 정리하면

|       | 자동화 스크립트로 만든 파이프라인 사례 | 최상길님이 실제로 할 방식            |
|-------|-----------------------|---------------------------|
| 실행 방식 | 개발자가 직접 코드 작성         | Claude Code 터미널에서 대화하듯 명령 |
| 호출 경로 | Console에서 발급받은 API 키  | 구독 계정 로그인                 |

|                      |                           |                                     |
|----------------------|---------------------------|-------------------------------------|
|                      | 자동화 스크립트로 만든 파이프라인 사례     | 최상길님이 실제로 할 방식                      |
| 과금                   | API 종량제 (별도 결제)           | 구독 한도 안에서 처리 (한도 초과 시에만 선택적 API 요금) |
| "100~200번 호출" 수치의 의미 | 이 파이프라인 개발자가 실측한 API 콜 횟수 | 참고용 — 페이지 몇 개가 영향받는지 감을 잡는 용도로만 활용  |

### 그래서 그 수치를 인용한 진짜 의도

제가 그 통계를 가져온 이유는 "이만큼 API 요금이 나온다"를 알려드리려던 게 아니라, **"소스 하나를 ingest하면 실제로 뒤에서 몇 번의 모델 호출/작업이 일어나는가"**라는 **작업량의 규모감**을 보여드리려던 것이었습니다. 하지만 "API 호출"이라는 단어 선택 때문에 마치 최상길님도 별도 API 요금을 내야 하는 것처럼 읽히게 됐네요. 죄송합니다.

**결론:** 최상길님이 Claude Code로 직접 ingest 명령을 내리시는 한, 이 "100~200번"에 해당하는 작업량은 그대로 구독 사용량으로 소진되는 것이지, 별도 API 청구서가 따로 날아오는 게 아닙니다. 다만 이 정도 규모의 작업이면 **구독의 5시간 세션 한도를 소스 하나 처리하는 데 상당 부분 써버릴 수 있다**는 감을 잡으시는 데는 여전히 유용한 참고치입니다.

### Q27. 사용자

```
C:\Users\chois>echo $ANTHROPIC_API_KEY
$ANTHROPIC_API_KEY
```

으로 출력되면 API Key가 설정된 상태야?

### A27. 어시스턴트(Claude)

좋은 질문입니다 — 그 결과는 **"API 키가 설정되어 있다"**는 뜻이 아니라, **명령어 자체가 잘못된 것**입니다. 하나씩 설명드리겠습니다.

#### 왜 이렇게 출력됐나

프롬프트가 `C:\Users\chois>`인 걸 보니 **cmd.exe(명령 프롬프트)**를 쓰고 계신 것 같습니다. `echo $ANTHROPIC_API_KEY`라는 명령은 사실 **macOS/Linux(bash/zsh)**나



**PowerShell 문법**이고, cmd.exe에서는 \$가 아무 의미가 없는 그냥 평범한 글자입니다. 그래서 cmd.exe는 "이게 환경변수구나"라고 해석하지 못하고, **입력하신 글자를 그대로 되돌려 출력한 것**입니다. 즉 이 결과는 "API 키가 없다"도 "있다"도 아니고, **애초에 확인이 이루어지지 않은 것**입니다.

### Windows 에서 올바르게 확인하는 방법

Windows에서 쓰시는 셸에 따라 문법이 다릅니다.

**cmd.exe(명령 프롬프트)를 쓰신다면:**

```
echo %ANTHROPIC_API_KEY%
```

\$ 대신 앞뒤로 %를 붙이는 게 cmd.exe의 환경변수 문법입니다.

**PowerShell을 쓰신다면:**

```
echo $env:ANTHROPIC_API_KEY
```

또는

```
$env:ANTHROPIC_API_KEY
```

### 결과 해석 방법

두 명령어 모두 결과는 둘 중 하나입니다:

- **아무것도 안 뜨고 빈 줄만 나옴** (또는 cmd.exe에서는 %ANTHROPIC\_API\_KEY%가 그대로 다시 출력됨) → 환경변수가 설정 안 되어 있다는 뜻 → **정상, 구독으로 과금됩니다**
- **`sk-ant-...`로 시작하는 긴 문자열이 출력됨** → 환경변수가 설정되어 있다는 뜻 → 이 경우 Claude Code가 구독을 무시하고 API로 과금하니, 지우시거나(unset) 의도한 게 맞는지 확인이 필요합니다

### 환경변수 지우는 방법 (혹시 설정되어 있었다면)

**cmd.exe (현재 세션만 임시로):**

```
set ANTHROPIC_API_KEY=
```

**cmd.exe (영구적으로, 시스템 환경변수에서 제거):**

```
setx ANTHROPIC_API_KEY ""
```

단, `setx`로 지우면 빈 문자열로 남을 수 있어서, 확실하게 지우려면 제어판 → 시스템 → 고급 시스템 설정 → 환경 변수에서 직접 `ANTHROPIC_API_KEY` 항목을 찾아 삭제하는 게 가장 확실합니다.

**PowerShell (현재 세션만 임시로):**

```
Remove-Item Env:ANTHROPIC_API_KEY
```

`%ANTHROPIC_API_KEY%(cmd)` 또는 `$env:ANTHROPIC_API_KEY(PowerShell)`로 다시 확인해서 아무것도 안 뜨면 정상적으로 제거된 것입니다.

## Q28. 사용자

너(Claude.ai)가 볼수 있을지 모르겠는데 내가 지금 my\_webpage라는 Claude code 프로젝트와 LLM\_Wiki\_DataScience 라는 프로젝트(이게 LLM Wiki이고 아직 초기 세팅 시작하려고해)를 운영하고 있어. web page 프로젝트내 자료실(실제 위치는 Github)에 내가 자료를 등록하면 이 자료가 LLM Wiki(나중에 이것도 github로 저장)의 Raw resource로 전달되어 자동으로 수집되게 하고 웹 페이지 상에서 자료실 자료에 대해 질의를 하면 그 질의가 LLM Wiki로 전달되어 답변을 받아 웹 페이지에 답변을 출력하는 식의 두 프로젝트 간의 연동 작업이 가능할까?

## A28. 어시스턴트(Claude)

좋은 아키텍처 질문이네요. 먼저 명확히 헤드릴 점: **저(Claude.ai 채팅)는 최상길님의 실제 GitHub 저장소나 Claude Code 프로젝트 파일에 직접 접근할 수는 없습니다.** 하지만 말씀하신 구조가 기술적으로 가능한지, 어떻게 설계해야 하는지는 설명해 드릴 수 있습니다.

결론부터 — 기술적으로 가능합니다, 다만 "그냥 되는" 게 아니라 별도 자동화 파이프라인을 만들어야 합니다

지금까지 안내드린 LLM Wiki 워크플로우는 **사람이 Claude Code 터미널에서 직접 대화하며 명령하는 방식**이었습니다. 그런데 지금 요청하신 건 **사람 개입 없이 두 프로젝트가 자동으로 신호를 주고받는 것**이라, 완전히 다른 층위의 작업입니다. 두 흐름을 나눠서 설계해야 합니다.

### 흐름 1 — 자료 등록 → LLM Wiki 자동 수집

```
[my_webpage 자료실에 자료 업로드]
  ↓ (GitHub push 이벤트)
[my_webpage 저장소의 GitHub Actions 트리거]
  ↓ (파일을 복사해서 LLM_Wiki_DataScience 저장소로 push)
[LLM_Wiki_DataScience 저장소의 raw/ 폴더에 자료 도착]
  ↓ (다시 GitHub Actions 트리거)
[헤드리스(비대화형)로 Claude를 호출해 ingest 수행]
  ↓
[wiki/ 폴더 업데이트 → 자동 커밋·푸시]
```

#### 핵심 부품:

- **GitHub Actions:** 자료실/ 폴더에 새 커밋이 생기면 자동으로 실행되는 워크플로우. `.github/workflows/*.yaml` 파일로 정의합니다.
- **저장소 간 전달:** my\_webpage 액션이 개인 액세스 토큰(PAT)을 이용해 LLM\_Wiki\_DataScience 저장소에 직접 파일을 push 합니다.
- **헤드리스 ingest:** 지금까지 쓰신 "Claude Code 터미널에서 대화하며 지시"하는 방식이 아니라, ``claude -p "이 파일 ingest 해줘"`` 같은 **비대화형(print mode) 명령**이나 Claude Agent SDK 로 스크립트를 짜서, 사람 개입 없이 자동으로 raw 폴더를 읽고 wiki 를 갱신하도록 만들어야 합니다.

### 흐름 2 — 웹페이지 질의 → LLM Wiki 답변 → 웹페이지 출력

```
[웹페이지에서 사용자가 질문 입력]
  ↓
[백엔드 함수(서버리스)가 질문을 받음]
  ↓
[LLM_Wiki_DataScience 저장소의 index.md + 관련 페이지를 읽어옴]
  ↓
[Claude API 호출 - 질문 + 위키 내용을 컨텍스트로 전달]
  ↓
```

[답변을 웹페이지로 반환해서 화면에 출력]

**여기서 중요한 함정:** 이전에 안내드린 개인 웹사이트는 **GitHub Pages(정적 호스팅)**로 배포되어 있습니다. 정적 호스팅은 백엔드 서버가 없기 때문에, "사용자 질문을 받아서 Claude API를 호출하는" 로직이 웹페이지 자체에는 들어갈 수 없습니다. API 키를 브라우저 코드에 그대로 넣으면 노출되어 도용당하기 때문에 절대 안 됩니다.

그래서 이 부분엔 **별도의 서버리스 백엔드**가 필요합니다. 예를 들면:

- Cloudflare Workers, Vercel Functions, AWS Lambda 같은 서버리스 함수
- 여기에 API 키를 "비밀 환경변수(secret)"로 안전하게 저장해두고, 웹페이지는 이 함수에만 요청을 보내고 함수가 대신 Claude API 를 호출

## 비용 측면 — 앞서 나눈 대화와 연결됩니다

이렇게 GitHub Actions나 서버리스 함수로 **자동화된 무인 파이프라인**을 만들면, 이견 더 이상 "사람이 로그인해서 구독으로 쓰는" 경로가 아니라 **API 키를 이용한 프로그램적 호출**이 됩니다. 즉 이전에 설명드린 "이중 지출" 구분에서 **이 자동화 파이프라인 부분은 구독이 아니라 API 종량제 요금으로 청구되는 영역**에 해당합니다. 자료 등록 빈도, 웹페이지 질의 빈도에 비례해서 이 부분만큼은 별도 API 비용이 실제로 발생한다고 보셔야 합니다.

## 정리 — 필요한 구성 요소

| 구성 요소                                      | 역할                                        |
|--------------------------------------------|-------------------------------------------|
| GitHub Actions (my_webpage)                | 자료실 업로드 감지 → LLM Wiki 저장소로 파일 전달          |
| GitHub Actions (LLM_Wiki_DataScience)      | 새 raw 파일 감지 → 헤드리스 ingest 실행 → wiki 자동 커밋 |
| Claude Agent SDK 또는 <code>claude -p</code> | 사람 개입 없이 ingest를 수행하는 헤드리스 실행 방식          |
| 서버리스 백엔드 (Cloudflare/Vercel 등)             | 웹페이지 질문을 받아 위키를 읽고 Claude API로 답변 생성      |
| API 키(비밀 환경변수)                             | 자동화 구간에서만 사용, 절대 프론트엔드 코드에 노출 금지          |

**요약하면:** 개념적으로 말씀하신 파이프라인은 충분히 구현 가능하고, 실제로 커뮤니티에서 비슷한 자동화 사례들이 나오고 있습니다. 다만 지금까지의 "대화형 Claude Code 사용"과는 다르게, **GitHub Actions 워크플로우 작성, 헤드리스 실행 스크립트, 서버리스 백엔드**라는 세 가지를 실제로 개발해야 하는 별도 엔지니어링 작업입니다.

이 정도 규모의 실제 구현 작업은 대화창보다 코드를 직접 짜고 실행하며 반복 검증하는 환경이 훨씬 잘 맞습니다.

## Q29. 사용자

---

그럼 이런건 두 프로젝트 중 어떤 프로젝트의 Claude code에 물어보고 진행하는게 좋을 까?

## A29. 어시스턴트(Claude)

---

좋은 실무적 질문입니다. 결론부터 말씀드리면 **"어느 한쪽에서만 다 해결한다"**기보다, **순서와 접근 방식을 나누는 게 낫습니다.**

### 왜 한쪽만으로는 어려운가

이 연동 작업은 정의상 두 저장소 사이에 걸쳐 있습니다:

- my\_webpage 쪽에 만들어야 할 것: 자료실 업로드 감지 → LLM Wiki 로 파일 전달하는 GitHub Actions
- LLM\_Wiki\_DataScience 쪽에 만들어야 할 것: raw 파일 수신 → 헤드리스 ingest 실행 → 질의 응답을 처리할 백엔드 로직

한쪽 프로젝트의 Claude Code 세션은 기본적으로 그 폴더 안의 파일만 보고 작업합니다. 그래서 어느 한쪽에서만 작업을 시작하면, 다른 쪽 저장소 구조나 규칙을 모른 채로 "일단 우리 쪽만 완성하고 상대는 나중에" 식이 되기 쉽습니다.

추천 순서 — LLM\_Wiki\_DataScience 부터 시작하시는 게 좋습니다

이유는 이렇습니다:

22. **LLM\_Wiki\_DataScience** 가 아직 초기 세팅 전 단계이기 때문에, 여기서 먼저 "이 위키가 외부로부터 자료를 어떻게 받을 것인지, 질의 요청을 어떤 형태로 받아서 어떻게 답할 것인지"에 대한 **규격(contract)**을 먼저 정의하는 게 순서상 맞습니다.
23. 이 규격이 먼저 나와야, my\_webpage 쪽에서 "그 규격에 맞춰서 파일을 보내는" GitHub Actions 를 제대로 짤 수 있습니다. 반대로 하면 my\_webpage 쪽 로직을 나중에 다시 뜯어고쳐야 할 가능성이 큼니다.

#### LLM\_Wiki\_DataScience의 Claude Code에게 먼저 물어볼 것:

- CLAUDE.md 스키마 작성 시 "자동화 파이프라인으로부터 raw 파일을 받는다"는 전제를 포함해서 설계
- 헤드리스(claude -p 등) ingest 를 트리거하는 GitHub Actions 워크플로우 작성
- 질의를 받아 답을 반환하는 백엔드 로직(어떤 입출력 형식을 쓸지)

#### 그 다음 my\_webpage의 Claude Code에게 물어볼 것:

- 자료실 업로드 감지 → LLM Wiki 저장소로 파일을 push 하는 GitHub Actions
- LLM Wiki 가 정해진 백엔드 엔드포인트로 질문을 보내고 답을 받아와서 화면에 출력하는 프론트엔드/서버리스 함수 코드

#### 만약 두 저장소를 동시에 다뤄야 하는 부분이 생기면

두 저장소가 로컬 디스크에서 형제 폴더 구조(~/.claude/projects/my\_webpage/, ~/.claude/projects/llm-wiki-datascience/ 같은)로 있으시다면, 그 상위 폴더에서 새 Claude Code 세션을 하나 열어서 두 저장소를 한 번에 보이게 하고 "연동 부분만" 작업하는 방법도 있습니다. 이렇게 하면 규격 불일치 문제 없이 한 번에 양쪽 워크플로우 파일을 일관되게 작성할 수 있습니다.

#### 정리

| 순서 | 어느 프로젝트 Claude Code  | 하는 일                                          |
|----|----------------------|-----------------------------------------------|
| 1  | LLM_Wiki_DataScience | 위키 초기 설정 + 외부 수신/응답 규격 정의 + 헤드리스 ingest 워크플로우 |
| 2  | my_webpage           | 규격에 맞춰 자료 전달 액션 + 질의 요청/답변 표시 로직              |

| 순서   | 어느 프로젝트 Claude Code   | 하는 일                             |
|------|-----------------------|----------------------------------|
| (선택) | 두 저장소를 함께 보는 상위 폴더 세션 | 연동 부분(양쪽 워크플로우 파일)을 한 번에 일관되게 작성 |

요약하면, "받는 쪽(LLM Wiki)"에서 규격을 먼저 확정하고, "보내는 쪽(my\_webpage)"이 거기에 맞추는 순서로 진행하시는 걸 추천드립니다.

### Q30. 사용자

웹페이지 질문을 받아 위키를 읽고 답변하는 기능이 없다면 API 비용이 발생하지는 않아요?

### A30. 어시스턴트(Claude)

좋은 질문입니다. 결론부터 말씀드리면, **질의응답 기능만 뺀다고 API 비용이 0이 되는 건 아닙니다.** 왜냐하면 앞서 설명드린 두 흐름 중 **자료 자동 수집(ingest)** 흐름 자체가 이미 **API 비용이 발생하는 구조**이기 때문입니다.

왜 그런가 — 비용이 발생하는 기준을 다시 짚어보면

이전에 말씀드린 기준은 "**사람이 로그인해서 대화하며 쓰는가**" vs "**자동화 스크립트가 API 키로 무인 호출하는가**"였습니다.

| 흐름               | 실행 방식                                     | 비용                      |
|------------------|-------------------------------------------|-------------------------|
| 자료 자동 수집(ingest) | GitHub Actions가 사람 없이 자동으로 헤드리스 Claude 호출 | <b>API 키 사용 → 비용 발생</b> |
| 웹페이지 질의응답        | 서버리스 백엔드가 사람 없이 자동으로 Claude API 호출        | <b>API 키 사용 → 비용 발생</b> |

두 흐름 다 "**무인 자동화**"라는 공통점이 있어서, **질의응답만 뺀다고 자료 수집 쪽 비용까지 사라지진 않습니다.** 자료실에 파일을 올릴 때마다 GitHub Actions가 자동으로 Claude를 불러서 ingest를 수행하는 이상, 그 호출 자체가 이미 API 종량제 청구 대상입니다.

그럼 비용이 0 이 되려면 어떻게 해야 하나

정말로 API 비용을 아예 안 내고 싶으시면, **자동화(GitHub Actions 트리거)를 없애고, ingest도 사람이 직접 Claude Code 터미널에서 명령해야 하는 구조로 되돌려야 합니다.**  
즉:

자동화 O (지금 설계) → 자료 올리면 자동으로 ingest 실행 → API 비용 발생  
자동화 X (수동 방식) → 자료를 raw/ 폴더에 넣어두고, 사람이 나중에 Claude Code 열어서 "수집해줘" 직접 명령 → 구독 사용량으로 처리, 추가 비용 없음

그러니까 질의응답 기능 유무는 "비용을 얼마나 더 낼지"의 문제이지 "비용이 나느냐 마느냐"의 문제가 아닙니다

정리하면 이렇게 4가지 조합이 나옵니다:

| 자동 ingest | 자동 질의응답        | 비용                                                                        |
|-----------|----------------|---------------------------------------------------------------------------|
| O         | O              | 두 흐름 다 API 비용 발생 (가장 비쌈)                                                  |
| O         | X (지금 질문하신 경우) | ingest 자동화 부분만 API 비용 발생, 질의응답은 없으니 그 부분 비용은 없음 — <b>비용이 줄긴 하지만 0은 아님</b> |
| X         | O              | 질의응답 부분만 API 비용 발생                                                        |
| X         | X              | 전부 사람이 Claude Code로 수동 진행 → 구독 사용량 안에서 해결, 추가 비용 없음                       |

실질적으로 얼마나 차이가 날지

두 흐름 중 어느 쪽이 더 비용이 큰지는 **호출 빈도와 처리량**에 달려 있습니다:

- **ingest**: 자료 하나당 여러 페이지를 다시 쓰는 작업이라 1 회당 비용이 큰 편이지만, 자료를 등록하는 빈도는 상대적으로 낮을 수 있음 (예: 하루 몇 건)
- **질의응답**: 1 회당 비용은 ingest 보다 작지만(관련 페이지 몇 개만 읽는 수준), 웹사이트 방문자가 자주 질문할수록 호출 빈도가 훨씬 높아질 수 있음

즉 방문자가 많은 웹사이트라면 질의응답 쪽을 빼는 게 오히려 비용 절감 효과가 더 클 수도 있고, 반대로 자료를 자주 대량으로 올리는 상황이라면 ingest 쪽 비용이 더 클 수도 있습니다. 정확한 판단을 위해서는 실제 예상 사용 빈도를 가지고 따져보시는 게 좋습니다.